UNIVERSIDAD AUTONOMA DE CHILE

Introducción a la Robótica

Diseña, imprime y programa tu propio robot seguidor de línea

Guía paso a paso: diseño CAD en OnShape, impresión 3D en PLA, electrónica con NodeMCU ESP8266, módulo L298N y sensores infrarrojos, programación en C++, control PID y manejo por Bluetooth desde el celular.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería

Primera edición · 2026

Introducción a la Robótica
Diseña, imprime y programa tu propio robot seguidor de línea

Primera edición, 2026.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería
Universidad Autónoma de Chile

Esta obra se distribuye bajo licencia
Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)

Eres libre de copiar, distribuir y adaptar este material citando la fuente, sin fines comerciales y compartiendo las obras derivadas bajo la misma licencia.

Los archivos del proyecto (código para el NodeMCU, app de App Inventor, modelos STL y este libro en PDF) están disponibles para descarga libre en el repositorio del libro:
`https://github.com/EduardoFuentesUA/IntroRobotica`

Compuesto con LaTeX (LuaLaTeX, clase KOMA-Script).
Tipografías: TeX Gyre Pagella, TeX Gyre Heros y TeX Gyre Adventor.

El logotipo de la Universidad Autónoma de Chile se reproduce con fines identificatorios. Los nombres de productos y marcas mencionados (Arduino, NodeMCU, Espressif ESP8266, OnShape, Ultimaker Cura, MIT App Inventor) pertenecen a sus respectivos titulares y se citan con fines educativos.

Índice general

0 Prólogo: ¿por qué construir un robot? **1**
0.1 ¿Qué es la robótica? 1
0.2 La robótica hoy 1
0.3 Qué construirás en este libro 2
0.4 Cómo usar este libro 2

1 Materiales y software **4**
1.1 Lista de materiales 4
1.2 Software que vas a instalar 6

2 Diseño CAD en OnShape **8**
2.1 ¿Qué es el CAD paramétrico? 8
2.2 La interfaz de OnShape 8
2.3 Croquis y restricciones 9
2.4 Operaciones básicas 9
2.5 Manos a la obra: diseña tu chasis 10
2.6 Exportar a STL 11
2.7 El chasis del autor 11

3 Impresión 3D **13**
3.1 Manufactura aditiva: qué es y por qué FDM 13
3.2 El material: PLA 13
3.3 Diseño para impresión 3D 14
3.4 Laminando en Cura 14
3.5 Manos a la obra: imprime tu chasis 15
3.6 Fallas típicas 15

4 Electrónica **17**
4.1 Conceptos base 17
4.2 Descripción de componentes 17
4.3 Por qué necesitas un puente H 17
4.4 Los sensores infrarrojos TCRT5000 19
4.5 Alimentación 20
4.6 Manos a la obra: cableado y ensamble 21

5 Programación **25**
5.1 ¿Qué es programar? 25
5.2 Prepara el Arduino IDE para el ESP8266 25
5.3 Estructura de un programa 26
5.4 De un motor a dos: funciones y tracción diferencial 27
5.5 Leer los sensores de línea 28
5.6 Manos a la obra: tu rutina de prueba 29
5.7 Depuración con el Monitor Serie 30

6 Comunicación **31**
6.1 ¿Qué es un protocolo, y por qué Bluetooth? 31
6.2 El módulo HC-05 y el puerto serial 31

6.3 El protocolo de mensajes del robot . . . 32
6.4 La app de control en MIT App Inventor . . . 34
6.5 Manos a la obra: empareja y controla tu robot . . . 35

7 Seguidor de línea y control PID . . . 37
7.1 Seguir una línea es un problema de control . . . 37
7.2 De los sensores a un número: el error . . . 37
7.3 Qué es un PID, con palabras simples . . . 38
7.4 El programa del seguidor . . . 39
7.5 Manos a la obra: pista y sintonía . . . 44

8 Epílogo . . . 46
8.1 Retos para tu pista . . . 46
8.2 Lo que aprendiste . . . 46
8.3 Hacia dónde seguir . . . 47

A Diagrama de conexiones . . . 48

B Guía de resolución de problemas . . . 50

C Archivos descargables . . . 54

0 Prólogo: ¿por qué construir un robot?

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es la robótica y por qué es un campo interdisciplinario.
- Reconocer robots que ya te rodean y situar al seguidor de línea como problema de aprendizaje.
- Saber qué construirás exactamente a lo largo de este libro.
- Conocer cómo está organizado el libro y cómo sacarle provecho.

0.1 ¿Qué es la robótica?

Un robot no es una sola disciplina: es el punto donde varias se encuentran. La **mecánica** le da un cuerpo capaz de moverse (ruedas, ejes, engranajes). La **electrónica** le entrega lógica y control (motores, sensores). La parte **eléctrica** resuelve cómo llega la energía desde la batería hasta cada componente sin que nada se queme en el camino. Y la **informática**, a través de un programa, decide *qué* hace el robot y *cuándo* lo hace.

Ningún robot funciona si falla una sola de estas cuatro capas. Un chasis perfecto con una batería mal conectada no se mueve. Un motor bien alimentado sin código no sabe qué hacer. Por eso la robótica se aprende mejor construyendo: cada capítulo de este libro resuelve una capa a la vez, y solo al final las cuatro conviven en la misma máquina.

NOTA

No necesitas saber nada de electrónica, CAD ni programación para empezar. Esta es una guía práctica que asume que no tienes experiencia previa.

0.2 La robótica hoy

Ya convives con robots todos los días, aunque no siempre los llames así: el brazo que suelda carrocerías en una fábrica de autos, el robot aspirador que esquiva los muebles de una casa, el dron que entrega un paquete. Todos comparten la misma estructura de fondo que vas a construir en este libro: sensores o entradas que informan del entorno, un controlador que decide, y actuadores que ejecutan esa decisión sobre el mundo físico. La Figura 0.1 muestra un ejemplo de cada contexto: la industria, el hogar y los vehículos que siguen una línea.

Para aprender robótica hace falta un problema concreto, no solo teoría. El **seguidor de línea** es un clásico: un vehículo que recorre solo un camino marcado en el suelo, igual que los vehículos guiados que mueven carga en fábricas y bodegas. Es un buen problema porque obliga a resolver percepción, control, movimiento y comunicación a la vez, con una meta simple de entender: no salirse de la línea. Es, además, el ejemplo perfecto para conocer el control **PID**, una de las

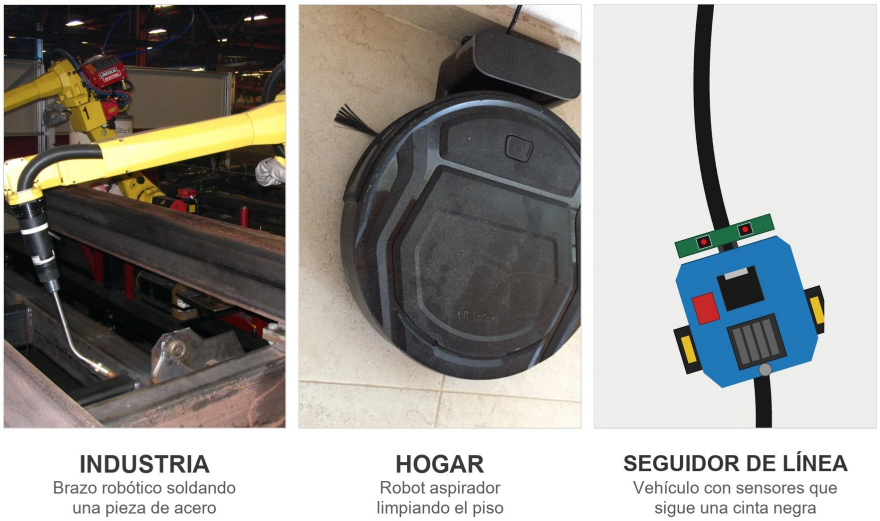


Figura 0.1. Robots presentes en distintos contextos: industria, hogar y vehículos guiados por una línea (de izquierda a derecha: un brazo robótico soldando, un robot aspirador y un robot seguidor de línea). Fotografías: Phasmatisnox (CC BY 3.0) y Air fans (CC0); el seguidor de línea es una ilustración propia; montaje y recortes propios.

ideas más usadas de la ingeniería de control, que aprenderás en el Capítulo 7 con un lenguaje sencillo.

0.3 Qué construirás en este libro

A lo largo de estos capítulos vas a diseñar, imprimir, cablear, programar y controlar un robot seguidor de línea: un vehículo de dos ruedas, cada una movida por su propio motor. Si una gira más rápido que la otra, el robot se tuerce hacia el lado de la más lenta; con esa idea simple se maneja todo el vehículo. Lleva además dos sensores infrarrojos que «ven» una cinta negra sobre el piso. Su cerebro es un NodeMCU, una placa pequeña basada en el chip ESP8266, que decide cuánto girar con un controlador PID. También lo puedes manejar a mano, de forma inalámbrica, por Bluetooth, desde una aplicación de celular hecha en MIT App Inventor que vas a instalar, entender por dentro y adaptar. La Figura 0.2 muestra el resultado.

CONSEJO PRÁCTICO

Si en algún capítulo no ves de inmediato para qué sirve lo que estás construyendo, vuelve a la Figura 0.2. Cada pieza del libro —cada motor, cada línea de código, cada conexión Bluetooth— existe para que este robot termine funcionando.

0.4 Cómo usar este libro

El libro está organizado como una secuencia de construcción, no como un manual de referencia: cada capítulo se apoya en el anterior y te acerca un paso más al robot terminado. Vas a encontrar siete elementos recurrentes que marcan el ritmo de la lectura:

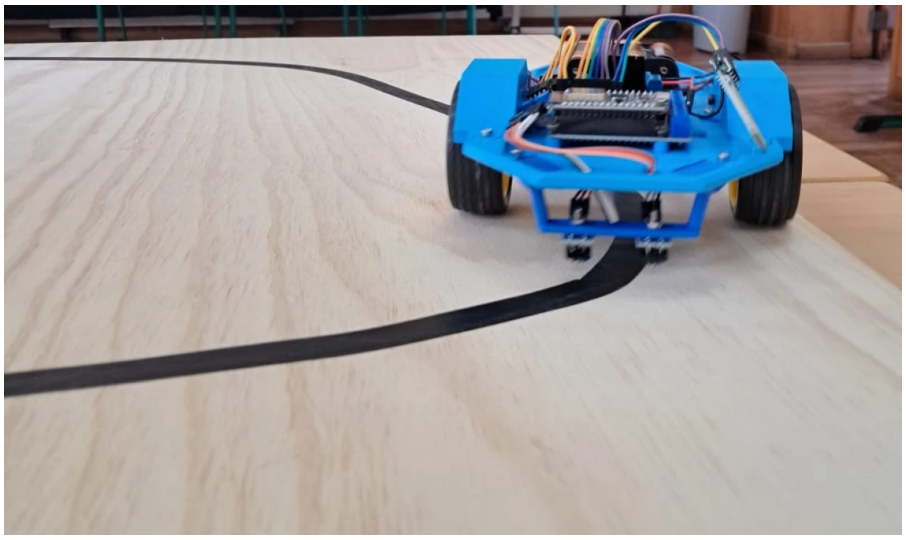


Figura 0.2. El robot seguidor de línea terminado sobre su pista, visto de frente: el módulo de sensores va bajo el borde delantero del chasis.

- Una caja de **objetivos** al abrir cada capítulo, como la de arriba.
- Cajas de **nota** con información complementaria.
- Cajas de **atención** cuando hay riesgo real de daño a un componente.
- Cajas de **consejo práctico** con atajos y errores comunes.
- Cajas de **manos a la obra** con la tarea concreta que ejecutas.
- Cajas de **antes de seguir, verifica** al cierre de cada sección clave.
- Un **resumen** al final de cada capítulo.

No necesitas ningún conocimiento previo de electrónica, CAD ni programación, ni tampoco herramientas más allá de las que se listan en el Capítulo 1. Lo que sí necesitas es continuar los capítulos en orden: cada uno da por hecho lo que construiste en el anterior.

RESUMEN DEL CAPÍTULO

- La robótica combina mecánica, eléctrica, electrónica e informática: ninguna capa funciona sola.
- Los robots que ya te rodean comparten la misma estructura —sensores, control, actuadores— que vas a construir en este libro.
- Vas a construir un robot seguidor de línea de dos ruedas motrices, con control PID y modo manual por Bluetooth desde tu celular.
- El libro se lee en orden: cada capítulo depende de lo construido en el anterior.

1 Materiales y software

EN ESTE CAPÍTULO APRENDERÁS

- Reunir todos los componentes y herramientas que vas a necesitar.
- Instalar y probar los cuatro programas que vas a usar en este libro.
- Llegar al Capítulo 2 con todo listo, sin interrupciones a mitad de camino.

Antes de tocar cualquier herramienta conviene reunir todo el material. Interrumpir el Capítulo 4 a mitad del armado porque falta un componente es la forma más común de perder el impulso de un proyecto. Este capítulo cierra ese riesgo.

1.1 Lista de materiales

La Tabla 1.1 reúne todo lo que necesitas para construir el robot completo. Ningún componente se usa antes del Capítulo 4, así que puedes seguir comprando o esperando envíos mientras avanzas con los Capítulos 2 y 3.

ATENCIÓN

El shield `HW-588A` solo calza con el NodeMCU angosto (versión «Amica», con chip `CP2102`). El NodeMCU «LoLin V3» es más ancho y no entra en sus conectores. Revisa la foto y las medidas antes de comprar.

NOTA

Muchos cables micro-USB baratos solo sirven para cargar: no llevan las líneas de datos y la computadora nunca detecta la placa. Si en el Capítulo 5 el NodeMCU no aparece, prueba primero con otro cable.

CONSEJO PRÁCTICO

Además de los componentes, necesitas herramientas básicas: un destornillador pequeño, un alicate de corte, y —muy recomendable aunque no obligatorio— un multímetro para verificar voltajes antes de conectar cada etapa nueva. Un soldador es opcional: todo el cableado de este libro se resuelve con borneras de tornillo y cables jumper, sin soldadura.

NOTA

No necesitas tener impresora 3D propia. El Capítulo 3 asume que vas a usar una impresora de acceso compartido (de tu universidad, un makerspace o un servicio de impresión); lo único tuyo es el filamento y el diseño.

Tabla 1.1. Materiales necesarios para el robot seguidor de línea.

Componente	Cant.	Rol en el robot
NodeMCU ESP8266 v1.0 (ESP-12E, chip USB `CP2102`)	1	Microcontrolador, cerebro del sistema
Shield `HW-588A` para NodeMCU	1	Base del NodeMCU: reparte la alimentación y trae pines para conectar todo sin soldar
Módulo puente H `L298N`	1	Control de potencia y sentido de los motores DC
Motor DC con reductora + rueda	2	Tracción diferencial (avance, retroceso, giro)
Módulo seguidor de línea, 2 canales independientes, con sensores infrarrojos `TCRT5000`	1	«Ojos» del robot: detectan la cinta negra bajo el chasis
Módulo Bluetooth `HC-05`	1	Enlace inalámbrico con la app del celular
Rueda loca / caster	1	Tercer punto de apoyo del chasis
Portapilas 4×AA con interruptor	1	Alimentación (6 V)
Pilas AA alcalinas	4	Energía para todo el robot
Capacitor electrolítico ≥470 µF, ≥16 V	1	Desacople del bus de alimentación: absorbe las caídas de tensión al arrancar los motores
Cables jumper hembra-hembra	~16	Señales entre el shield, el `L298N`, los sensores y el `HC-05`
Cable rojo y negro para borneras	1 m	Alimentación entre pilas, `L298N` y shield
Cable USB a micro-USB, con datos	1	Programar el NodeMCU desde tu computadora
Celular Android	1	Corre la app de control (Capítulo 6)
Filamento PLA	1 rollo	Impresión del chasis (Capítulo 3)
Cinta aislante negra, de unos 19 mm de ancho	1 rollo	La línea que seguirá el robot (Capítulo 7)
Cartulina o papel blanco, o un piso claro y liso	~1 m^2	Superficie de la pista: contrasta con la cinta

1.2 Software que vas a instalar

Vas a usar cuatro programas a lo largo del libro. Ninguno es gratuito por casualidad: los cuatro tienen una versión gratuita o educativa suficiente para este proyecto.

OnShape
: CAD paramétrico que corre en el navegador, sin instalación. Lo usas en el Capítulo 2 para diseñar el chasis del robot.

Ultimaker Cura
: Convierte tu diseño 3D en instrucciones que entiende la impresora (*slicing*). Lo usas en el Capítulo 3.

Arduino IDE
: Entorno donde escribes y cargas el código al NodeMCU. Aunque la placa no es un Arduino, se programa igual: basta agregarle al IDE el paquete del ESP8266 (Capítulo 5). Lo usas desde el Capítulo 5 en adelante.

MIT App Inventor
: Herramienta web para construir aplicaciones de celular. En el Capítulo 6 abres en ella la app que controla el robot.

La Figura 1.1 muestra los cuatro programas juntos.

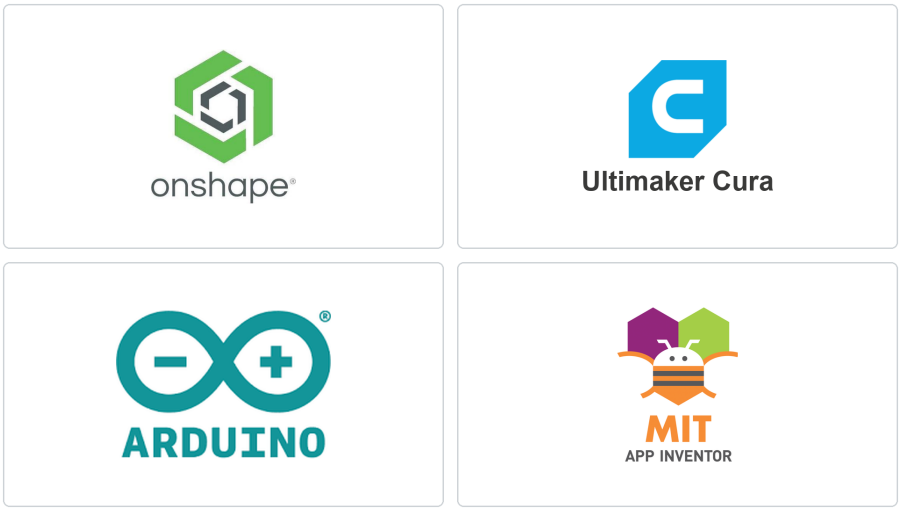


Figura 1.1. Los cuatro programas que usarás en este libro: OnShape, Ultimaker Cura, Arduino IDE y MIT App Inventor.

MANOS A LA OBRA

Prepara tu equipo antes de continuar al Capítulo 2:

1. Crea una cuenta gratuita en `onshape.com` (solo necesitas correo).
2. Descarga e instala Ultimaker Cura desde su sitio oficial.
3. Descarga e instala el Arduino IDE desde `arduino.cc`. El paquete del ESP8266 lo agregas en el Capítulo 5.
4. Crea una cuenta gratuita en `appinventor.mit.edu` (con cuenta Google).
5. Abre los cuatro programas al menos una vez y confirma que inician sin errores.

ANTES DE SEGUIR, VERIFICA

- [ ] Tienes en mano (o en camino) todos los componentes de la Tabla 1.1.
- [ ] Tu NodeMCU es la versión angosta, compatible con el shield `HW-588A`.
- [ ] Tu cuenta de OnShape está creada y abre el editor.
- [ ] Cura y el Arduino IDE están instalados y abren sin errores.
- [ ] Tu cuenta de MIT App Inventor está creada.

RESUMEN DEL CAPÍTULO

- El robot necesita un microcontrolador (NodeMCU ESP8266) montado en su shield, dos motores DC, un módulo de dos sensores infrarrojos, un puente H, un módulo Bluetooth y una fuente de alimentación: nada de eso se usa antes del Capítulo 4, pero conviene tenerlo listo desde ahora.
- Vas a usar cuatro programas, uno por etapa del proyecto: OnShape (diseño), Cura (impresión), Arduino IDE (programación) y MIT App Inventor (comunicación).
- No necesitas soldador ni impresora 3D propia para seguir este libro.

2 Diseño CAD en OnShape

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es el CAD paramétrico y por qué lo vas a usar.
- Moverte con soltura en la interfaz de OnShape.
- Crear un croquis con restricciones y convertirlo en una pieza sólida.
- Diseñar el chasis de tu robot y exportarlo listo para imprimir.

2.1 ¿Qué es el CAD paramétrico?

CAD (diseño asistido por computadora) es la herramienta con la que un ingeniero dibuja una pieza antes de fabricarla. La palabra clave es **paramétrico**: cada medida de tu diseño queda guardada como un número que puedes cambiar después. Si dibujas un agujero de 5 mm y luego necesitas 6 mm, no rehaces el dibujo: cambias el número.

Vas a usar **OnShape** porque corre en el navegador —sin instalación ni licencia— y porque cualquier cambio se guarda solo, en la nube.

2.2 La interfaz de OnShape

Al abrir un documento nuevo vas a ver tres zonas: el árbol de *features* a la izquierda (el historial de todo lo que has hecho), el área de trabajo en el centro, y la barra de herramientas de croquis y operaciones arriba (Figura 2.1).

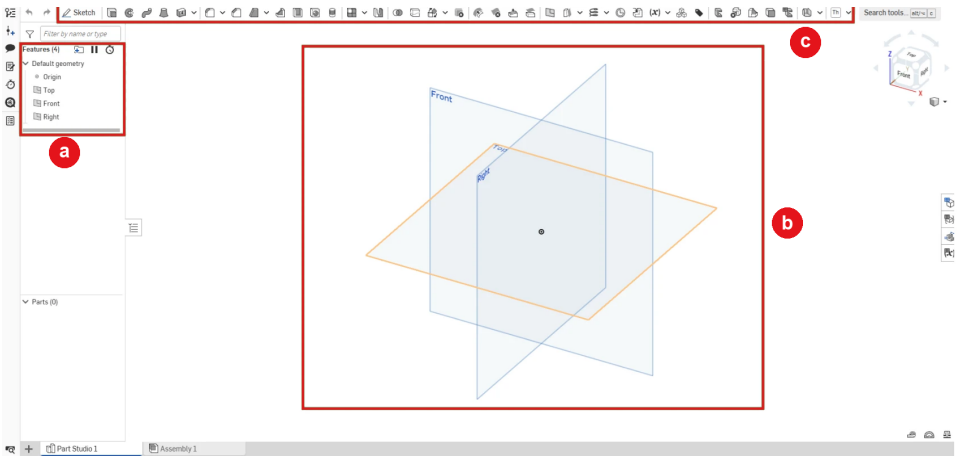


Figura 2.1. Interfaz de OnShape: árbol de features (a), área de trabajo (b) y barra de herramientas (c).

NOTA

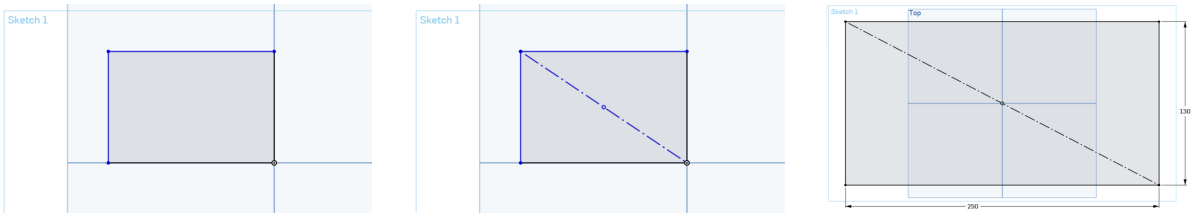
Cada pestaña en la parte inferior es un *Part Studio* o un *Assembly*. En este capítulo trabajas siempre dentro de un único Part Studio: ahí vive el chasis completo como una sola pieza.

2.3 Croquis y restricciones

Todo sólido en OnShape empieza como un croquis plano (*sketch*) dibujado sobre una cara o un plano de referencia.

1. Selecciona el plano superior (*Top*) y entra en modo croquis (*Sketch*).
2. Dibuja un rectángulo con la herramienta *Corner rectangle*.
3. Acota el ancho y el alto con la herramienta *Dimension*: escribe el valor de cada cota.
4. Centra el rectángulo en el origen: dibuja una diagonal entre dos esquinas opuestas, conviértela en línea de construcción (*Construction*) y aplica la restricción *Midpoint* entre el origen y esa diagonal. Así el rectángulo queda centrado aunque cambies las medidas después.
5. Confirma que el croquis está completamente restringido: OnShape lo dibuja en negro. Si queda algo en azul, falta una restricción o una cota.

La Figura 2.2 muestra el croquis en tres momentos: recién dibujado, al aplicar la restricción *Midpoint* y totalmente restringido.



(**a**) Rectángulo inicial (**b**) Restricción de punto medio (**c**) Croquis totalmente restringido

Figura 2.2. Creación de un croquis base con restricciones.

CONSEJO PRÁCTICO

Si un croquis se ve azul y no sabes qué falta, arrastra con el mouse uno de los puntos azules: lo que se mueve es lo que aún no está restringido. Si algo se ve en rojo, hay restricciones en conflicto. Cuando el croquis se complica, el *Constraint Manager* (una herramienta de diagnóstico del croquis) lista las restricciones y te ayuda a encontrar la que falta o la que sobra. Aprender a leer los colores te va a servir en cada pieza del chasis.

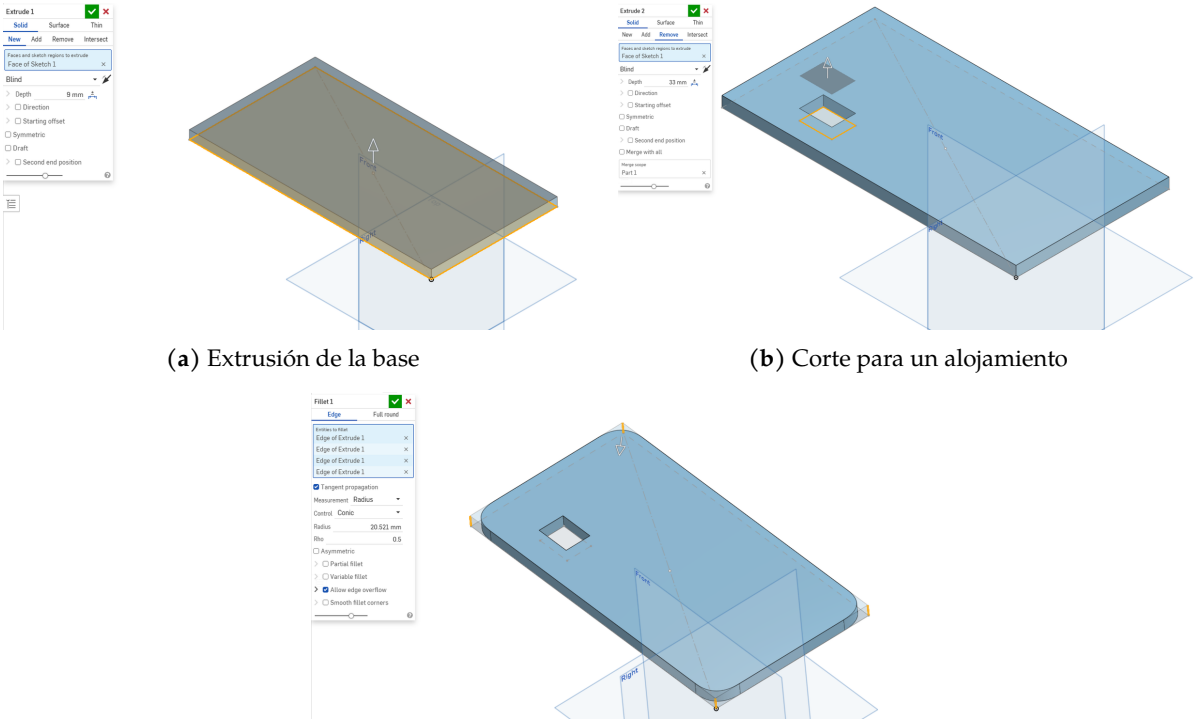
2.4 Operaciones básicas

Con el croquis cerrado, tres operaciones te alcanzan para la mayoría de las piezas de este proyecto:

1. **Extrusión (Extrude):** convierte el croquis plano en un sólido con espesor. Es la operación que más vas a usar.

2. **Corte (Extrude con la opción Remove):** quita material siguiendo un croquis nuevo dibujado sobre una cara existente. Así se hacen los agujeros para tornillos y las ranuras de sujeción de los componentes.
3. **Redondeo (Fillet):** suaviza una arista. Además de verse mejor, una arista redondeada imprime en 3D con menos problemas que una esquina viva.

La Figura 2.3 muestra las tres operaciones aplicadas sobre la misma placa.



(**a**) Extrusión de la base

(**b**) Corte para un alojamiento

(**c**) Redondeo de aristas

Figura 2.3. Las tres operaciones básicas aplicadas sobre la placa base.

2.5 Manos a la obra: diseña tu chasis

MANOS A LA OBRA

1. Mide físicamente cada componente de la Tabla 1.1 (Capítulo 1): NodeMCU montado en su shield `HW-588A`, módulo `L298N`, motores con reductora, módulo de sensores `TCRT5000`, módulo `HC-05`, portapilas y rueda caster.
2. Decide dónde va cada componente sobre una placa base. Deja espacio para el cableado y para que el portapilas se pueda retirar sin desarmar nada. Mantén la placa dentro del volumen de impresión de tu impresora (en muchas es de unos 220×220 mm).
3. Dibuja un boceto a mano de la vista superior, con las medidas que acabas de tomar. No lo saltes: dibujar a mano obliga a decidir la distribución antes de pelear con el mouse.
4. Recién ahora, traslada ese boceto a un croquis en OnShape y extrúyelo en la placa base.
5. Agrega los cortes para fijar cada componente (ranuras para el módulo `L298N`, orificios para el shield del NodeMCU). Deja a la vista el conector USB del NodeMCU: vas a enchufarlo cada vez que cargues un programa.
6. Diseña el soporte de los sensores, en el borde delantero de la placa y con las reglas de

la nota siguiente.

NOTA

Los sensores mandan sobre el diseño del frente. Los dos sensores `TCRT5000` van en el borde delantero, adelantados respecto del eje de las ruedas, mirando hacia el suelo y a unos 5–10 mm de él. Sepáralos ~10–20 mm más que el ancho de la cinta (para una cinta de 19 mm, unos 30–40 mm entre sus centros), de modo que con el robot centrado la cinta pase justo entre los dos. Hazles ranuras alargadas en vez de agujeros redondos: así podrás subir o bajar el módulo sin reimprimir. Verifica estas medidas en tu croquis antes de extruir.

2.6 Exportar a STL

1. En el árbol de features, selecciona la pieza terminada.
2. Haz clic derecho y elige *Export*.
3. Elige formato `STL`, resolución *Fine*, y descarga el archivo.

NOTA

Guarda el archivo con un nombre claro, por ejemplo `chasis-robot.stl`. Lo vas a necesitar abierto en Cura en el Capítulo 3.

2.7 El chasis del autor

La Figura 2.4 muestra el chasis que se usó para escribir este libro. Tómalo como referencia de proporciones y ubicación de componentes, no como una plantilla obligatoria: tu propio diseño puede —y probablemente debería— verse distinto. Sus archivos STL están en el repositorio del libro (Anexo C).

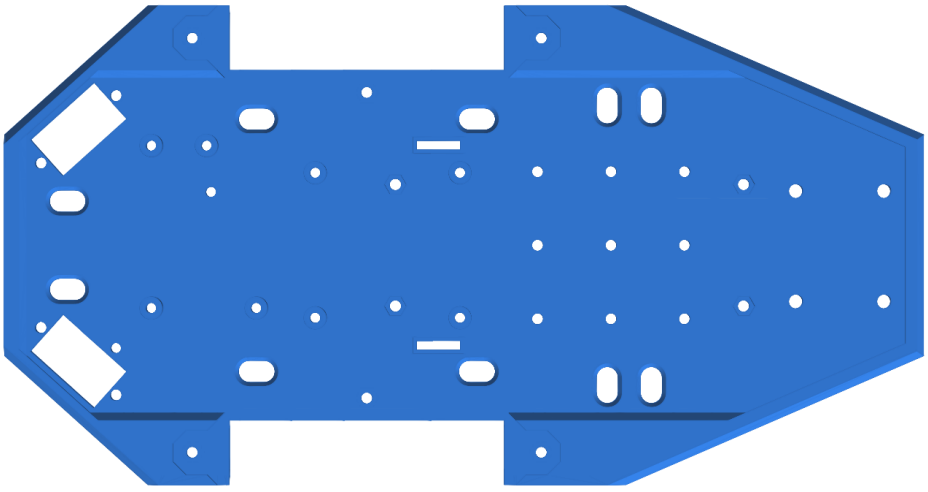


Figura 2.4. Placa del chasis de referencia del autor, vista desde arriba (250×130 mm).

ANTES DE SEGUIR, VERIFICA

- [ ] Tu croquis base está completamente restringido (en negro, sin azul).
- [ ] El chasis tiene espacio marcado para cada componente de la Tabla 1.1 que va montado en el robot.
- [ ] El soporte de sensores deja los dos `TCRT5000` al frente, mirando al suelo, separados según el ancho de tu cinta y con ranuras para regular su altura.
- [ ] Exportaste un archivo `.stl` y lo tienes guardado.

RESUMEN DEL CAPÍTULO

- El CAD paramétrico guarda cada medida como un número editable: por eso conviene restringir bien los croquis desde el principio.
- Extrusión, corte y redondeo alcanzan para diseñar la mayoría de las piezas de este proyecto.
- El chasis se diseña a partir de mediciones reales de tus componentes, no al revés.
- La posición de los sensores (altura, separación y distancia al eje) es una restricción de diseño, no un detalle estético: de ella depende que el robot vea la línea.

3 Impresión 3D

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es la manufactura aditiva y cómo imprime una impresora FDM.
- Conocer el PLA: sus temperaturas, ventajas y límites.
- Preparar tu chasis en Ultimaker Cura y convertirlo en instrucciones de impresión.
- Imprimir el chasis que diseñaste en el Capítulo 2.

3.1 Manufactura aditiva: qué es y por qué FDM

Fabricar una pieza por **manufactura sustractiva** significa partir de un bloque de material y quitarle lo que sobra —como tallar o fresar—. Fabricarla por **manufactura aditiva** significa lo contrario: construirla agregando material capa por capa, desde cero, hasta completar la forma.

La tecnología que vas a usar es **FDM** (Modelado por Deposición Fundida): un filamento sólido se funde en una boquilla caliente y se deposita capa sobre capa, siguiendo exactamente el camino que definiste al diseñar el chasis en OnShape. En la Figura 3.1 se ven las tres partes que importan: la boquilla, la cama caliente y los ejes de movimiento.

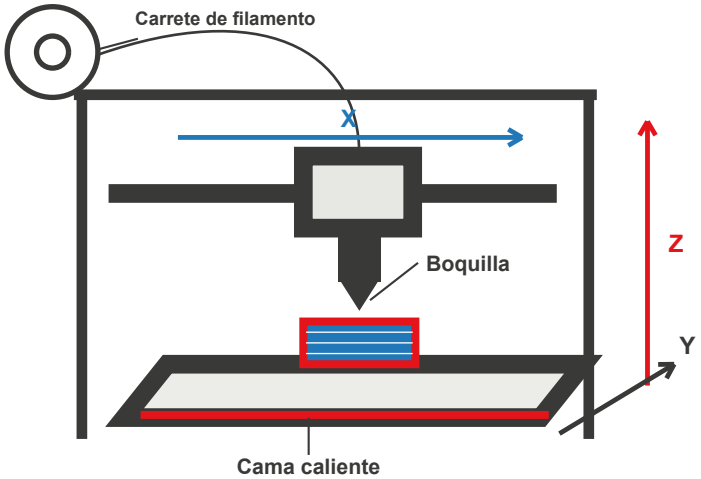


Figura 3.1. Anatomía de una impresora FDM: boquilla, cama caliente y ejes de movimiento.

3.2 El material: PLA

Vas a imprimir en **PLA** (ácido poliláctico), el material más usado para empezar en impresión 3D: es fácil de imprimir, no necesita cama caliente a temperaturas extremas y no desprende gases fuertes.

Tabla 3.1. PLA comparado con otros filamentos comunes.

Material	Boquilla	Cama	Para este proyecto
PLA	190–220 °C	50–60 °C	Recomendado: fácil, rígido, suficiente para el chasis
PETG	230–250 °C	70–80 °C	Más resistente al impacto, más difícil de imprimir bien
ABS	220–250 °C	90–110 °C	Requiere recinto cerrado; se deforma (*warping*) sin él

NOTA

El PLA es más frágil que el PETG o el ABS a temperatura ambiente, pero para un chasis que no queda expuesto al sol ni a temperaturas altas, esa diferencia no importa. Empieza con PLA; puedes reimprimir en otro material más adelante si una pieza en particular lo necesita.

3.3 Diseño para impresión 3D

Antes de laminar el chasis del Capítulo 2, revisa estos cuatro puntos: una impresora FDM no fabrica cualquier geometría igual de bien.

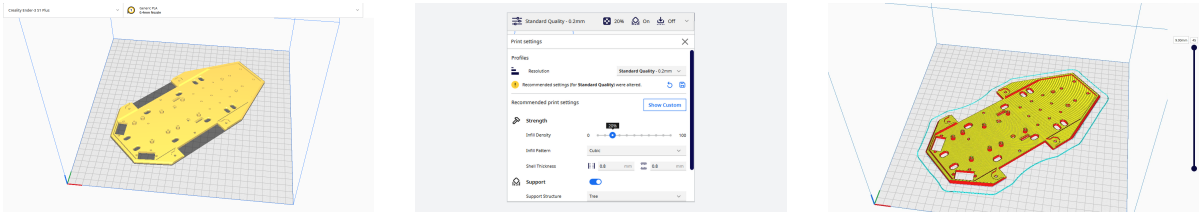
- **Voladizos:** una superficie que sobresale en el aire sin apoyo necesita soporte, o se cae mientras se imprime. Si puedes evitarlo reorientando la pieza, hazlo.
- **Orientación:** la pieza es más fuerte a lo largo de las capas que entre ellas. Orienta el chasis para que las fuerzas del uso normal —el peso de los motores, un choque contra el borde de la pista— no tiendan a separar capas.
- **Anisotropía:** por lo anterior, una pieza impresa en 3D nunca es igual de resistente en todas direcciones. No es un defecto de tu impresión: es una propiedad del proceso.
- **Agujeros y tornillos:** un agujero impreso sale ligeramente más angosto que su medida nominal. Diseña los agujeros para tornillos con 0.2–0.3 mm de holgura extra. Cada impresora y cada filamento se portan distinto: antes de imprimir el chasis, haz una pieza de prueba con agujeros de 0.1, 0.2 y 0.3 mm de holgura y quédate con la medida que mejor ajuste.

3.4 Laminando en Cura

Laminar (*slicing*) es convertir tu modelo 3D en instrucciones capa por capa que la impresora entiende: el código G.

1. Abre Cura, elige el perfil de tu impresora e importa el archivo `chasis-robot.stl` que exportaste en el Capítulo 2. Si el modelo aparece con rayas grises, se sale del volumen de impresión: vuelve a OnShape y redúcelo.
2. Selecciona el perfil de material PLA y ajusta la altura de capa (0.2 mm es un buen punto de partida: buen acabado sin imprimir demasiado lento).
3. Define el relleno (*Infill Density*; 20 % alcanza para el chasis) y activa los soportes (*Support*) solo en los voladizos que identificaste en la sección anterior. En la figura se usaron soportes de tipo árbol (*Tree*), que gastan menos material y se quitan con facilidad.
4. Revisa la adherencia a la cama (*Adhesion*). El chasis es una placa plana con mucha superficie de contacto, así que en la figura se dejó desactivada; si en tu impresora las esquinas se levantan, activa un borde (*brim*).
5. Exporta el código G a la tarjeta SD o envíalo directo a la impresora.

La Figura 3.2 muestra el modelo importado, los ajustes y la vista previa por capas.



(**a**) Modelo importado (**b**) Ajustes de impresión (**c**) Vista previa por capas

Figura 3.2. Preparación del chasis en Ultimaker Cura (perfil Creality Ender-3 S1 Plus, PLA genérico, capa de 0.2 mm, relleno de 20 % y soportes de tipo árbol; la interfaz está en inglés).

CONSEJO PRÁCTICO

La vista previa por capas de Cura simula la impresión completa antes de gastar filamento. Revísala siempre: si ves soportes flotando en el aire o una base de contacto muy pequeña, corrige la orientación antes de imprimir, no después.

3.5 Manos a la obra: imprime tu chasis

MANOS A LA OBRA

1. Carga el código G de tu chasis en la impresora.
2. Nivela la cama si la impresora lo requiere, siguiendo su manual.
3. Inicia la impresión y supervisa las primeras capas: son las que definen si la pieza se pega bien a la base.
4. Cuando termine, retira la pieza con cuidado y quita los soportes con un alicate o una espátula.

3.6 Fallas típicas

Tabla 3.2. Diagnóstico rápido de fallas comunes en impresión FDM.

Síntoma	Causa probable	Solución
Las esquinas se levantan de la cama (*warping*)	Mala adherencia o enfriamiento desigual	Usa *brim*, limpia la cama, evita corrientes de aire
Hilos finos entre piezas (*stringing*)	Temperatura muy alta o falta de retracción	Baja 5–10 °C la boquilla, activa retracción
La pieza se despega a mitad de impresión	Cama sucia, sin nivelar, o primera capa muy rápida	Limpia con alcohol isopropílico, nivela, reduce velocidad de la primera capa

ANTES DE SEGUIR, VERIFICA

- ☐ Identificaste los voladizos de tu chasis y decidiste su orientación.
- ☐ El chasis cabe en el volumen de impresión: en Cura no aparece con rayas grises.
- ☐ Laminaste el modelo en Cura con altura de capa 0.2 mm y relleno 20 %.
- ☐ Revisaste la vista previa por capas antes de imprimir.
- ☐ Tu chasis está impreso, sin soportes, y encaja con las medidas que tomaste en el Capítulo 2.

RESUMEN DEL CAPÍTULO

- La manufactura aditiva construye piezas capa por capa; la FDM lo hace fundiendo filamento.
- El PLA es el material correcto para empezar: fácil de imprimir y suficiente para este chasis.
- Orientación, voladizos y anisotropía son decisiones de diseño, no detalles de la impresora.
- La mayoría de las fallas de impresión se resuelven en la primera capa o en la temperatura, no cambiando de máquina.

4 Electrónica

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué hace cada componente eléctrico y electrónico del robot.
- Saber por qué un motor DC nunca se conecta directo al NodeMCU.
- Armar la alimentación del robot sin quemar ningún componente.
- Cablear y montar todos los componentes sobre tu chasis, en etapas que puedes verificar una por una.

Este es el capítulo donde más fácil es dañar un componente. No es motivo para tener miedo, sino para ir por etapas: cada sección de armado más adelante trae su propia verificación, y no vas a energizar nada que no puedas revisar antes.

4.1 Conceptos base

Tres cantidades explican casi todo lo que vas a hacer en este capítulo. El **voltaje** es la presión eléctrica disponible, en voltios (V). La **corriente** es cuánta electricidad realmente fluye, en amperios (A) — un motor detenido a la fuerza pide más corriente, no más voltaje. La **resistencia**, en ohmios (Ω), es lo que se opone a ese flujo. Y **GND** (tierra o masa) es el punto de referencia común: todo voltaje se mide *respecto a* GND, así que todos los GND del robot deben estar conectados entre sí.

4.2 Descripción de componentes

Los componentes que forman el robot están reunidos en la Tabla 4.1: qué hace cada uno y el dato que más te conviene recordar. Las secciones siguientes los explican uno por uno.

NOTA

El shield `HW-588A` trae su propio controlador de motores (un `L293D`), con borneras marcadas `A` y `B`. En este robot no se usa: entrega apenas 0.6 A por canal, y el `L298N` soporta bastante más. Las borneras `A`, `B` y `VM` del shield quedan libres; lo que sí usas son sus pines y su bornera de alimentación `VIN`.

4.3 Por qué necesitas un puente H

Tabla 4.1. Los componentes centrales de este capítulo.

Componente	Qué hace	Dato clave
Motor DC + reductora	Convierte electricidad en giro; la reductora baja las RPM y multiplica el torque	Pide cientos de mA al arrancar
Módulo de sensores `TCRT5000` (2 canales)	Cada canal ilumina el piso con luz infrarroja y mide cuánta vuelve: distingue la cinta negra del fondo blanco	Salida digital por canal (`HIGH` o `LOW`); trabaja a 3.3 V
Módulo `L298N`	Puente H: controla sentido y velocidad de motores DC que piden más corriente de la que entrega el microcontrolador	Hasta ~2 A por canal; cae ~2 V
NodeMCU ESP8266	Microcontrolador: ejecuta el programa y genera las señales de control. Trae WiFi, que este robot no usa	Trabaja a 3.3 V; cada pin entrega solo ~12 mA
Shield `HW-588A`	Base donde se enchufa el NodeMCU: reparte la alimentación y da pines para conectar todo sin soldar	Recibe entre 4.5 y 9 V por su bornera `VIN`

ATENCIÓN

Nunca conectes un motor DC directamente a un pin del NodeMCU. El pin entrega apenas unos 12 mA a 3.3 V, y el motor demanda cientos de miliamperios en el arranque. Vas a quemar el pin, y en el peor caso, el microcontrolador completo.

El módulo `L298N` resuelve esto: el NodeMCU le manda una señal de control de apenas unos miliamperios, y el `L298N` —con su propia alimentación— mueve la corriente real que el motor necesita. Además, invierte la polaridad del motor según la señal, lo que te permite elegir el sentido de giro sin tocar cables.

La velocidad se controla con **PWM** (Modulación por Ancho de Pulso): en vez de entregar un voltaje constante, el pin enciende y apaga muchas veces por segundo. Cuanto más tiempo pasa encendido en cada ciclo, más rápido gira el motor. `ENA` y `ENB` son los pines del `L298N` que reciben esa señal PWM, uno por motor.

Tabla 4.2. Conexión entre el NodeMCU (pines del shield) y el módulo L298N.

NodeMCU	L298N	Función
`D8`	`ENA`	Velocidad del motor izquierdo (PWM)
`D7`	`IN1`	Sentido del motor izquierdo
`D6`	`IN2`	Sentido del motor izquierdo
`D3`	`ENB`	Velocidad del motor derecho (PWM)
`D5`	`IN3`	Sentido del motor derecho
`D4`	`IN4`	Sentido del motor derecho
`GND`	`GND`	Masa común, obligatoria

NOTA

El NodeMCU trabaja a 3.3 V y el `L298N` a 5 V. No hace falta adaptar nada: el `L298N` reconoce como `HIGH` cualquier señal sobre unos 2.3 V, así que los 3.3 V del NodeMCU le bastan. Lo mismo vale para el `HC-05`.

ATENCIÓN

El `L298N` viene de fábrica con un jumper sobre `ENA` y otro sobre `ENB`, que dejan los motores siempre a velocidad máxima. **Retíralos** antes de conectar los cables de `D8` y `D3`: si los dejas, el NodeMCU no puede regular la velocidad.

NOTA

`D3`, `D4` y `D8` cumplen además un papel al encender el ESP8266: de su nivel depende que arranque tu programa o que espere uno nuevo. Con el `L298N` funcionan sin problema, pero si algún día el NodeMCU no arranca con todo conectado, desconecta esos tres cables y prueba de nuevo (Anexo B).

4.4 Los sensores infrarrojos TCRT5000

Tu robot necesita «ver» la línea. Para eso usa un módulo con dos canales independientes, cada uno con un sensor `TCRT5000`: un diodo emisor de luz infrarroja (invisible para ti) y un fototransistor que mide cuánta de esa luz rebota de vuelta. Una superficie blanca refleja casi toda la luz; la cinta negra la absorbe. Como la luz que vuelve cambia, el sensor distingue una de otra (Figura 4.1).

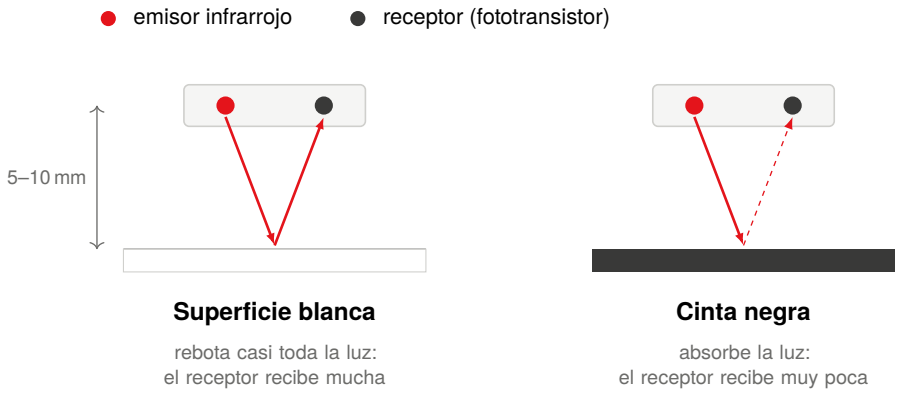


Figura 4.1. Cómo distingue un sensor `TCRT5000` la cinta negra de una superficie clara.

El módulo no entrega esa luz tal cual: trae un pequeño comparador (un chip `LM393`) que la convierte en una señal digital, `HIGH` o `LOW`, y un **potenciómetro** con el que ajustas el umbral —qué tanta luz cuenta como «blanco»—. Cada canal tiene además un LED indicador que cambia de estado cuando el sensor pasa de una superficie a la otra, y tres pines: dos de alimentación y la salida digital (revisa en tu módulo cómo está rotulada).

Tabla 4.3. Conexión entre el NodeMCU (pines del shield) y el módulo de sensores.

NodeMCU	Módulo	Función
`D1`	Salida del canal izquierdo	Entrega `HIGH` o `LOW` según lo que ve
`D0`	Salida del canal derecho	Ídem, para el lado derecho
`3V`	`VCC` de ambos canales	Alimentación a 3.3 V
`GND`	`GND` de ambos canales	Masa común, obligatoria

ATENCIÓN

Alimenta los sensores con los **3.3 V** del NodeMCU (pin `3V`), no con `VIN`. Un módulo alimentado a 5 V entrega señales de 5 V, y los pines del ESP8266 no las soportan: pueden dañarse. A 3.3 V el módulo funciona bien.

NOTA

Con los módulos de este libro, la salida es `LOW` sobre la superficie clara y `HIGH` sobre la cinta negra (o cuando no vuelve luz, por ejemplo si levantas el robot). Otros módulos pueden ser al revés: por eso, en el Capítulo 5, el programa trae una constante que invierte esa lógica con un solo cambio.

NOTA

En el shield, el pin `D1` puede llegar además a su controlador de motores interno. Como sus borneras `A` y `B` no tienen nada conectado, la señal del sensor no tiene ningún efecto ahí.

4.5 Alimentación

Tu robot se alimenta con **4 pilas AA en serie**, que entregan ~6 V, en un portapilas con interruptor. Toda esa energía entra primero al `L298N`, y desde ahí se reparte:

1. Las pilas van a la bornera de entrada del `L298N` (marcada `12V`) y a su `GND`. Con eso el módulo mueve los motores.
2. Con su jumper de 5 V puesto, el `L298N` enciende un regulador interno que entrega una tensión estable en su bornera `5V`.
3. Esa salida va a la bornera `VIN` del shield, junto con `GND`. Desde `VIN` se alimentan el NodeMCU —que la baja a 3.3 V con su propio regulador— y el módulo `HC-05`. Los sensores, en cambio, toman sus 3.3 V del pin `3V` del NodeMCU.

NOTA

El `L298N` pierde ~2 V al controlar los motores: con 6 V de entrada, los motores reciben ~4 V, no los 6 V nominales. Es normal, y conviene saberlo para no esperar más torque o velocidad de la que el sistema realmente entrega.

NOTA

El regulador del `L298N` necesita unos 7 V de entrada para entregar 5 V exactos. Con 4 pilas entrega algo menos, entre 4 y 5 V: alcanza para el NodeMCU y el `HC-05`, pero baja a medida que se gastan las pilas. Por eso este robot usa **pilas alcalinas nuevas**. Cuatro pilas recargables NiMH entregan solo 4.8 V en total, y con ellas el NodeMCU no recibe lo que necesita.

ATENCIÓN

En este montaje, el NodeMCU y el `HC-05` comparten la alimentación con la que se mueven los motores: `VIN` del shield viene de la salida de 5 V del `L298N`, que a su vez sale de las mismas pilas. Cuando un motor arranca o cambia de sentido, pide un pico de corriente que puede bajar esa tensión y reiniciar el NodeMCU. Instala un capacitor electrolítico de al menos 470 µF (16 V o más) entre `VIN` y `GND` del shield, con la polaridad correcta. **El capacitor no aumenta la capacidad del regulador.** Mide la tensión en `VIN` mientras los motores arrancan y se detienen; si el NodeMCU se reinicia, cambia las pilas, y si sigue pasando con pilas nuevas, mira el consejo siguiente.

CONSEJO PRÁCTICO

Con 4 pilas el margen es justo: el shield pide al menos 4.5 V en `VIN` y el regulador del `L298N` entrega entre 4 y 5 V. En el robot del autor, con pilas nuevas, funciona. Si el tuyo se reinicia aun así, sube la tensión de entrada: 6 pilas AA (9 V) o una batería LiPo 2S (7.4 V, con su cargador) dejan al regulador entregar 5 V reales. Revisa antes que tu `L298N` admita esa tensión con el jumper de 5 V puesto (muchos indican hasta 12 V). Los motores recibirán unos 2 V más: si el robot va demasiado rápido, baja `VELOCIDAD_BASE`.

ATENCIÓN

GND común obligatorio. Las pilas, el `L298N`, el shield con el NodeMCU, los sensores y el `HC-05` deben compartir la misma referencia de tierra. Sin GND común, las señales de control pierden su punto de referencia y el robot se comporta de forma errática o no responde.

CONSEJO PRÁCTICO

Las pilas AA alcalinas pierden voltaje rápido bajo el consumo de dos motores, y ese voltaje caído se nota como pérdida de fuerza a medida que avanza la sesión de pruebas. Si notas que el robot rinde menos después de media hora, o que el NodeMCU se reinicia al arrancar, no es tu cableado: son las pilas. Ten siempre un juego nuevo a mano. Y recuerda que un robot que cambia de velocidad a medida que se gastan las pilas también cambia la forma en que sigue la línea.

4.6 Manos a la obra: cableado y ensamble

Arma el robot en las siguientes etapas, en orden. Cada una trae su propia verificación: no avances a la siguiente sin pasarla. La Figura 4.2 muestra el esquema completo del cableado; consúltala en cada etapa. El módulo `HC-05` se conecta en el Capítulo 6.

MANOS A LA OBRA

1. **Portapilas al L298N.** Con el interruptor apagado, conecta el cable rojo del portapilas a la bornera `12V` del `L298N` y el negro a su `GND`. *Prueba:* enciende el interruptor; el LED del módulo se ilumina.
2. **Salida regulada del L298N.** Confirma que el jumper de 5 V está puesto. *Prueba:* con el multímetro, mide entre 4 y 5 V entre la bornera `5V` del módulo y `GND`.
3. **Shield y NodeMCU.** Apaga el interruptor. Enchufa el NodeMCU en el shield, con la antena y el conector USB hacia donde indica la serigrafía. Lleva un cable desde la bornera `5V` del `L298N` a la bornera `VIN` del shield, y otro de `GND` a `GND`. *Prueba:* al encender, el LED azul del NodeMCU parpadea un instante, y entre `VIN` y `GND` del shield mides lo mismo que en el paso anterior.
4. **Motores al L298N.** Conecta el motor izquierdo a `OUT1/OUT2` y el derecho a `OUT3/OUT4`. *Prueba:* con el multímetro en modo continuidad, confirma que no hay corto entre las dos salidas de un mismo motor.
5. **Señales de control.** Retira los jumpers de `ENA` y `ENB` del `L298N` y cablea, con jumpers hembra-hembra, `ENA`, `IN1`–`IN4` y `ENB` según la Tabla 4.2. *Prueba:* revisa cable por cable contra la tabla; un pin cruzado aquí es el error más común de todo el capítulo.
6. **Capacitor y sensores.** Instala el capacitor entre `VIN` y `GND` del shield, con su polo positivo a `VIN`. Conecta el módulo de sensores según la Tabla 4.3: `VCC` al pin `3V` —nunca a `VIN`—, `GND` a `GND` y la salida de cada canal a `D1` (izquierdo) y a `D0` (derecho). Si solo hay un pin `3V` libre, comparte la alimentación de ambos canales con un conector doble o una mini protoboard. *Prueba:* verifica cada cable contra la Figura 4.2; con el interruptor encendido, pasa una hoja blanca y luego la cinta negra bajo cada canal: su LED indicador debe cambiar de estado. Si no cambia, gira el potenciómetro con un destornillador pequeño hasta que lo haga.
7. **Verificación de GND común.** Con todo cableado y el interruptor apagado, mide continuidad entre el `GND` del `L298N`, el `GND` del shield y el `GND` del módulo de sensores. *Prueba:* el multímetro debe marcar continuidad (~0 Ω) entre los tres puntos.
8. **Montaje sobre el chasis.** Fija cada componente en su lugar en el chasis impreso en el Capítulo 3, con el cableado ordenado, sin tensión en los conectores y con el USB del NodeMCU accesible.

ANTES DE SEGUIR, VERIFICA

- ☐ El LED del módulo `L298N` enciende al conectar las pilas.
- ☐ La salida de 5 V del módulo mide entre 4 y 5 V con el multímetro.
- ☐ El NodeMCU enciende montado en el shield, alimentado por `VIN`.
- ☐ Los jumpers de `ENA` y `ENB` están retirados y cada pin de la Tabla 4.2 está cableado en el lugar correcto.
- ☐ El capacitor de desacople está instalado entre `VIN` y `GND`.
- ☐ Los sensores toman 3.3 V de `3V`, sus salidas llegan a `D1` y `D0`, y el LED de cada canal cambia al pasar de blanco a negro.
- ☐ Hay continuidad de GND entre `L298N`, shield y sensores.
- ☐ Todos los componentes están fijos sobre el chasis.

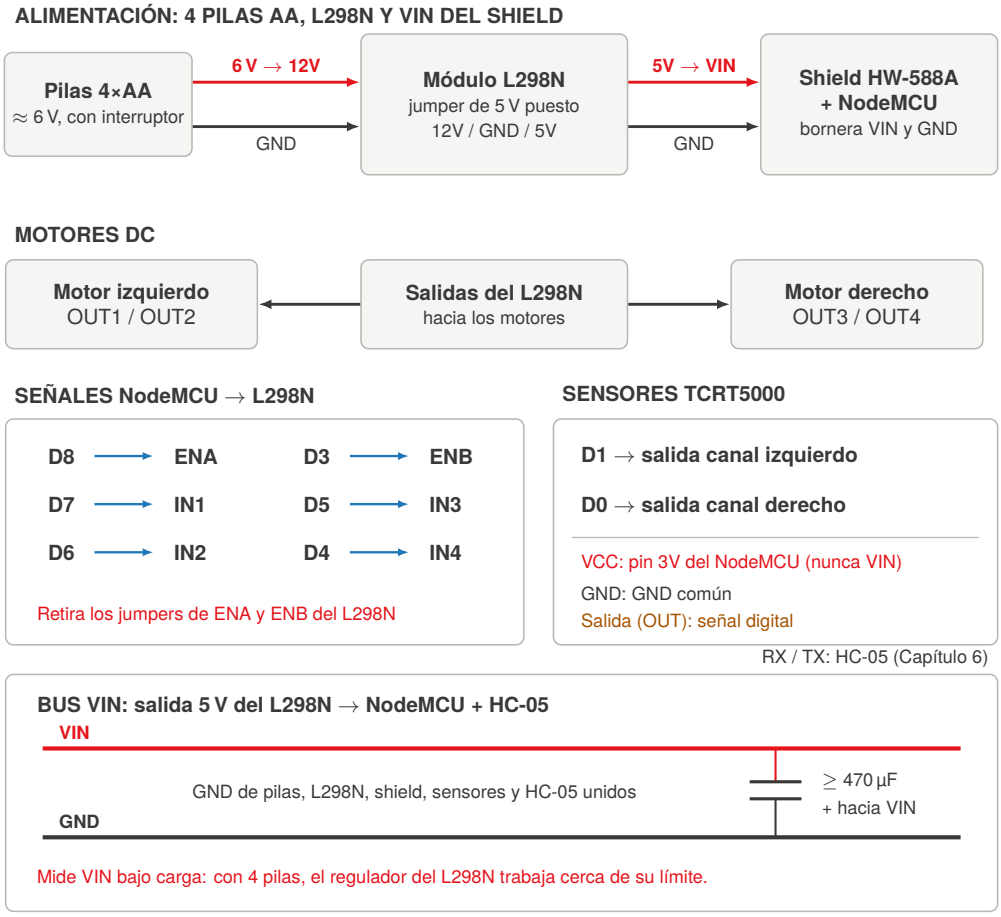


Figura 4.2. Esquema de conexión completo: pilas, L298N, shield con el NodeMCU, motores y módulo de sensores.

RESUMEN DEL CAPÍTULO

- Un motor DC nunca se conecta directo al NodeMCU: el `L298N` existe para manejar esa corriente.
- PWM controla la velocidad de los motores: cuanto más tiempo está encendido el pulso en cada ciclo, más rápido gira la rueda.
- Un sensor `TCRT5000` mide cuánta luz infrarroja rebota en el piso y la convierte en una señal digital: así distingue la cinta negra del fondo claro. Se alimenta a 3.3 V, nunca a 5 V.
- Las 4 pilas alimentan el `L298N`; su salida regulada llega al `VIN` del shield y de ahí al NodeMCU y el `HC-05`. Hay que verificar esa tensión bajo carga y usar pilas en buen estado.
- GND común no es opcional: sin él, el robot se comporta de forma errática aunque todo lo demás esté bien cableado.
- El armado en etapas verificables evita que un error de cableado aparezca recién cuando todo está montado y es difícil de revisar.

5 Programación

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es programar y por qué el NodeMCU se programa en C++.
- Preparar el Arduino IDE para programar el ESP8266.
- Reconocer la estructura de un programa: `setup()` y `loop()`.
- Escribir funciones que muevan un motor, luego dos, y leer los dos sensores de línea.
- Cargar una rutina de prueba completa y verificar que todo el robot responde.

5.1 ¿Qué es programar?

Programar es escribir instrucciones que una máquina ejecuta en orden, una tras otra. Un lenguaje de programación es *de bajo nivel* cuando esas instrucciones están cerca de lo que el procesador entiende directamente, y *de alto nivel* cuando están más cerca de cómo piensa una persona.

Tu NodeMCU se programa en **C++**, un lenguaje de nivel medio: te deja controlar cada pin directamente, pero con una sintaxis más cómoda que el ensamblador. El programa corre solo sobre el chip, sin sistema operativo de por medio, y C++ te da ese control fino sin desperdiciar memoria ni tiempo. Aunque el NodeMCU no es un Arduino, usa las mismas instrucciones: todo lo que aprendas aquí sirve también para una placa Arduino.

5.2 Prepara el Arduino IDE para el ESP8266

El Arduino IDE no trae de fábrica las herramientas para el ESP8266: se agregan una sola vez, como un paquete.

MANOS A LA OBRA

1. En el Arduino IDE abre *Archivo* → *Preferencias* y, en «Gestor de URLs adicionales de tarjetas», pega esta dirección: `https://arduino.esp8266.com/stable/package_esp8266com_index.json`
2. Abre *Herramientas* → *Placa* → *Gestor de tarjetas*, busca `esp8266` e instala el paquete «esp8266 by ESP8266 Community».
3. En *Herramientas* → *Placa*, elige **NodeMCU 1.0 (ESP-12E Module)**.
4. Conecta el NodeMCU por USB y elige su puerto en *Herramientas* → *Puerto*. Si no aparece ninguno, instala el controlador `CP2102` de Silicon Labs y prueba con otro cable: muchos solo cargan, no transmiten datos.

NOTA

Mientras programas, deja apagado el interruptor de las pilas: el USB alimenta el NodeMCU por sí solo. Enciende las pilas solo cuando el programa necesite mover los motores.

5.3 Estructura de un programa

Todo programa del Arduino IDE tiene dos funciones obligatorias:

- `setup()` se ejecuta una sola vez, al encender o reiniciar. Aquí configuras los pines.
- `loop()` se repite sin parar mientras la placa tenga corriente. Aquí vive el comportamiento del robot.

El Código 5.1 usa las tres instrucciones más básicas: `pinMode` declara si un pin es entrada o salida, `digitalWrite` lo pone en `HIGH` o `LOW`, y una variable (`const int LED`) le da nombre a un número de pin para no repetirlo por todo el código.

```
// Enciende y apaga el LED integrado del NodeMCU cada segundo.
// En el ESP8266 este LED funciona al revés: LOW lo enciende, HIGH lo apaga.
const int LED = LED_BUILTIN;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, LOW);   // encendido
  delay(1000);
  digitalWrite(LED, HIGH);  // apagado
  delay(1000);
}
```

Código 5.1. Parpadeo del LED integrado del NodeMCU.

NOTA

El LED integrado del NodeMCU está conectado «al revés»: se enciende con `LOW` y se apaga con `HIGH`. No es un error del programa; es cómo está cableado en la placa. Ese mismo pin es `D4`, que en tu robot también va al `L298N`: haz esta prueba con el interruptor de las pilas apagado.

NOTA

`analogWrite` funciona distinto de `digitalWrite`: en vez de `HIGH/LOW`, recibe un número y genera una señal PWM. Es la instrucción que vas a usar para controlar la velocidad de los motores. En el ESP8266, la escala de ese número cambió entre versiones del paquete (0 a 255 en las nuevas, 0 a 1023 en las antiguas); por eso los programas del libro llaman a `analogWriteRange(255)` en `setup()` y trabajan siempre de 0 a 255.

5.4 De un motor a dos: funciones y tracción diferencial

Una **función** le pone nombre a un bloque de instrucciones que vas a repetir. Sin funciones, mover dos motores en cuatro combinaciones de sentido —avanzar, retroceder, girar a cada lado— te obliga a repetir el mismo patrón de `digitalWrite` una y otra vez, con más oportunidades de equivocarte en algún cable.

El Código 5.2 define una función `moverMotor()` que recibe qué pines usar, a qué velocidad y en qué sentido, y encima construye `avanzar()`, `retroceder()`, `girarIzquierda()` y `girarDerecha()` llamando a esa función una vez por motor. Los pines se nombran como están impresos en la placa (`D8`, `D7`…): el paquete del ESP8266 ya sabe a qué patita del chip corresponde cada uno. Los dos motores están montados enfrentados, y sus cables quedan al revés uno del otro: por eso, para el motor derecho, la función recibe sus pines `IN3` e `IN4` en orden invertido.

```
// Control de los dos motores DC con el módulo L298N (ver Tabla 4.2, Capítulo 4).
const int ENA = D8, IN1 = D7, IN2 = D6;  // Motor izquierdo
const int ENB = D3, IN3 = D5, IN4 = D4;  // Motor derecho

// Mueve un motor hacia adelante o hacia atrás, con velocidad de 0 a 255.
// En el motor izquierdo, IN1 en LOW e IN2 en HIGH hacen avanzar la rueda. El
// motor derecho está cableado al revés (IN3 en HIGH e IN4 en LOW), así que
// a esta función se le pasan sus pines invertidos.
void moverMotor(int in1, int in2, int en, int velocidad, bool adelante) {
  digitalWrite(in1, adelante ? LOW : HIGH);
  digitalWrite(in2, adelante ? HIGH : LOW);
  analogWrite(en, velocidad);
}

void avanzar(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, true);
  moverMotor(IN4, IN3, ENB, velocidad, true);
}

void retroceder(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, false);
  moverMotor(IN4, IN3, ENB, velocidad, false);
}

void girarIzquierda(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, false);
  moverMotor(IN4, IN3, ENB, velocidad, true);
}

void girarDerecha(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, true);
  moverMotor(IN4, IN3, ENB, velocidad, false);
}

void detener() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

void setup() {
  analogWriteRange(255);  // velocidades de 0 a 255, igual que en un Arduino
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(IN3, OUTPUT);
```

```
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
}

void loop() {
  avanzar(150);
  delay(1000);
  detener();
  delay(500);
}
```

Código 5.2. Funciones de movimiento sobre el módulo L298N.

CONSEJO PRÁCTICO

Si tu robot avanza cuando debería retroceder, no cambies el código: lo más probable es que un motor esté cableado al revés. Intercambia sus dos cables en la bornera del `L298N` antes de tocar una sola línea de `moverMotor()`.

5.5 Leer los sensores de línea

Un motor es una *salida*: tu programa le dice qué hacer. Un sensor es lo contrario, una *entrada*: el programa le pregunta qué ve. Para leer un pin de entrada se declara con `pinMode(pin, INPUT)` y se consulta con `digitalRead(pin)`, que devuelve `HIGH` o `LOW`.

El Código 5.3 lee los dos sensores que conectaste a `D1` (izquierdo) y `D0` (derecho) y escribe en el Monitor Serie qué ve cada uno. Los módulos de este libro entregan `HIGH` sobre la cinta negra y `LOW` sobre la superficie clara, pero otros vienen al revés: la constante `NIVEL_LINEA` guarda el valor que corresponde a la cinta, y la función `sobreLinea()` lo compara: así, el resto del programa solo pregunta «¿estás sobre la línea?» y no depende de qué módulo compraste.

```
// Lectura de los dos sensores infrarrojos TCRT5000 (ver Tabla 4.3, Capítulo 4).
// Muestra en el Monitor Serie (9600 baudios) qué ve cada sensor.
const int SENSOR_IZQ = D1, SENSOR_DER = D0;

// Nivel que entrega el módulo sobre la cinta negra. Los módulos del libro dan
// HIGH sobre la cinta y LOW sobre la superficie clara. Si el tuyo es al revés,
// cambia HIGH por LOW y no toques nada más.
const int NIVEL_LINEA = HIGH;

bool sobreLinea(int pin) {
  return digitalRead(pin) == NIVEL_LINEA;
}

void setup() {
  Serial.begin(9600);
  pinMode(SENSOR_IZQ, INPUT);
  pinMode(SENSOR_DER, INPUT);
}

void loop() {
  Serial.print("Izquierdo: ");
  Serial.print(sobreLinea(SENSOR_IZQ) ? "LINEA" : "blanco");
  Serial.print("   Derecho: ");
  Serial.println(sobreLinea(SENSOR_DER) ? "LINEA" : "blanco");
  delay(200);
```

```
26 }
```

Código 5.3. Lectura de los dos sensores de línea.

MANOS A LA OBRA

Calibra tus sensores.

1. Sube el Código 5.3 y abre el Monitor Serie a 9600 baudios.
2. Apoya el robot sobre la superficie blanca de tu pista, con los sensores a la altura de trabajo (unos 5–10 mm del piso). Ambos deben decir `blanco`.
3. Desliza una tira de cinta negra bajo el sensor izquierdo, luego bajo el derecho. Cada uno debe decir `LINEA` solo mientras está sobre la cinta. Si dice `LINEA` en el blanco y `blanco` en la cinta, cambia `NIVEL_LINEA` de `HIGH` a `LOW`.
4. Si un sensor no cambia nunca, o cambia a media distancia, gira el potenciómetro de su canal con un destornillador pequeño hasta que acierte con el blanco y con la cinta a la altura de trabajo.

CONSEJO PRÁCTICO

La luz del sol y las lámparas de luz cálida contienen mucha luz infrarroja y pueden confundir al sensor. Si las lecturas cambian cuando abres una cortina, calibra de nuevo en el lugar donde vas a usar el robot.

5.6 Manos a la obra: tu rutina de prueba

MANOS A LA OBRA

1. Crea un programa nuevo en el Arduino IDE que junte lo que ya tienes: las funciones de movimiento del Código 5.2 y la lectura de los sensores del Código 5.3, con `NIVEL_LINEA` ya ajustado. En `loop()`, haz que el robot avance, retroceda, gire a la izquierda y gire a la derecha, con una pausa entre movimientos, y que al final informe por el Monitor Serie qué ve cada sensor mientras le pasas la cinta por debajo. Escribe un mensaje con `Serial.println()` antes de cada paso, para saber en qué punto está.
2. Conecta el NodeMCU a tu computadora por USB y súbelo con el botón de carga.
3. Con el robot sobre una superficie donde las ruedas giren libres, enciende el interruptor de las pilas.
4. Observa el ciclo completo: avanza, retrocede, gira a la izquierda, gira a la derecha y, al final, durante cinco segundos, informa por el Monitor Serie qué ve cada sensor mientras le pasas la cinta por debajo.
5. Mientras los motores arrancan y se detienen, mide la tensión entre `VIN` y `GND` del shield, como pedía el Capítulo 4.

ATENCIÓN

Si alguna función no responde —un motor no gira, un sensor siempre dice lo mismo— **no insistas con el código**. Vuelve al Capítulo 4 y revisa esa etapa específica del armado: en la enorme mayoría de los casos, un motor o sensor mudo es un cable en el pin equivocado o una alimentación mal conectada, no un error de programación.

5.7 Depuración con el Monitor Serie

La rutina de prueba llama a `Serial.begin(9600)` en `setup()`, y a `Serial.println()` antes de cada movimiento. Abre el Monitor Serie del Arduino IDE, a 9600 baudios y con el NodeMCU conectado por USB, para ver en texto qué función se está ejecutando en cada momento.

NOTA

Justo al encender, el ESP8266 envía por el puerto serial un mensaje de arranque a otra velocidad, que en el Monitor Serie aparece como una línea de caracteres extraños. Es normal: tu programa empieza a escribir después.

CONSEJO PRÁCTICO

El Monitor Serie es la herramienta más simple para saber si tu programa llegó a una parte del código: si el mensaje «Girar izquierda» nunca aparece, el problema está antes de esa línea, no después.

ANTES DE SEGUIR, VERIFICA

- [ ] El Arduino IDE tiene instalado el paquete del ESP8266 y reconoce el puerto del NodeMCU.
- [ ] El LED integrado del NodeMCU parpadea al cargar el Código 5.1.
- [ ] El robot avanza y retrocede en línea recta al ejecutar `avanzar()` y `retroceder()`.
- [ ] El robot gira hacia cada lado al ejecutar `girarIzquierda()` y `girarDerecha()`.
- [ ] Cada sensor dice `LINEA` sobre la cinta negra y `blanco` sobre la superficie clara, con `NIVEL_LINEA` ya ajustado.
- [ ] El Monitor Serie muestra el nombre de cada movimiento mientras el robot lo ejecuta.

RESUMEN DEL CAPÍTULO

- El NodeMCU se programa desde el Arduino IDE, en C++, después de instalar una vez el paquete del ESP8266.
- Todo programa tiene `setup()` (una vez) y `loop()` (siempre).
- Las funciones evitan repetir código y hacen visibles los errores de cableado: si `avanzar()` falla, sabes exactamente qué revisar.
- La tracción diferencial se logra combinando el sentido de giro de los dos motores, no con hardware adicional.
- Un sensor es una entrada: se lee con `digitalRead()`, y una constante (`NIVEL_LINEA`) aísla las diferencias entre módulos.
- El Monitor Serie es tu primera herramienta de diagnóstico, antes de sospechar del cableado.

6 Comunicación

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es un protocolo de comunicación y dónde encaja Bluetooth.
- Conectar el módulo HC-05 al NodeMCU sin dañarlo.
- Conocer el protocolo de comandos de tu robot y programarlo para recibirlos.
- Instalar la app de control hecha en MIT App Inventor, entender cómo está construida y adaptarla.

6.1 ¿Qué es un protocolo, y por qué Bluetooth?

Un **protocolo** es un acuerdo: dos dispositivos deciden de antemano qué significa cada dato que se envían, igual que dos personas que hablan el mismo idioma para poder entenderse. Esa conversación puede ir por cable o sin él. Con cable, los chips de un mismo circuito se hablan mediante `UART`, `I2C` o `SPI`, y tu computadora se comunica con el NodeMCU por el cable USB. Sin cable, los dispositivos se comunican por radio, con protocolos como WiFi y **Bluetooth**.

Tu NodeMCU trae WiFi integrado, pero no Bluetooth: el chip ESP8266 no tiene radio Bluetooth (otros chips, como el ESP32, sí). Para controlar el robot desde una app de MIT App Inventor el camino más directo es agregarle un módulo Bluetooth: todo celular lo trae, y el módulo que lo resuelve —el HC-05— es barato y se conecta con cuatro cables.

6.2 El módulo HC-05 y el puerto serial

El **HC-05** es un módulo Bluetooth que convierte lo que envías por comunicación serial en una señal Bluetooth, y viceversa. Desde el punto de vista del NodeMCU, hablar con el HC-05 es igual de simple que hablar con el Monitor Serie del Capítulo 5: de hecho, usa el mismo puerto.

En este robot el HC-05 va conectado a los pines seriales de hardware del NodeMCU, `RX` y `TX`. Es la conexión más simple: no necesita ninguna librería, y el programa lee los comandos con el mismo `Serial` que ya usaste para depurar.

ATENCIÓN

Los pines `RX` y `TX` son los mismos que usa el USB para cargar el programa. **Desconecta el HC-05 —o al menos su cable `TXD`— cada vez que subas un programa nuevo**, y vuelve a conectarlo después. Si lo dejas, la carga falla con un error de conexión, aunque el código esté perfecto.

Tabla 6.1. Conexión entre el NodeMCU (pines del shield) y el módulo HC-05.

NodeMCU	HC-05	Función
`RX`	`TXD`	El HC-05 transmite, el NodeMCU recibe
`TX`	`RXD`	El NodeMCU transmite, el HC-05 recibe
`VIN`	`VCC`	Alimentación del módulo
`GND`	`GND`	Masa común, obligatoria

NOTA

Las líneas de datos van **cruzadas**: lo que uno transmite (`TX`) es lo que el otro recibe (`RX`). Conectar `TX` con `TXD` es el error más común de esta etapa, y no daña nada, pero tampoco funciona. Los pines `EN` y `STATE` del HC-05 quedan sin conectar.

NOTA

Con una placa Arduino de 5 V haría falta un divisor de tensión antes de la entrada `RXD` del módulo, que espera 3.3 V. Con el NodeMCU no: trabaja a 3.3 V, igual que el HC-05, y los dos se conectan directo. La alimentación `VCC` sí va a `VIN`, porque el módulo acepta entre 3.6 y 6 V.

6.3 El protocolo de mensajes del robot

El protocolo lo define la app de control. Cada mensaje es una línea de texto que termina en un salto de línea, y el NodeMCU decide qué hacer según cómo empieza. Hay dos tipos (Tabla 6.2).

Tabla 6.2. Mensajes que manda la app Guia_Robotica.

Mensaje	Ejemplo	Qué hace
`M:izq,der`	`M:150,-150`	**Modo manual.** Fija la velocidad de cada motor, de -255 (atrás a fondo) a 255 (adelante a fondo).
`KP:x,KI:y,KD:z`	`KP:400,KI:0,KD:0`	**Modo automático.** Fija las ganancias del PID y pone en marcha el seguidor de línea (Capítulo 7).

En el mensaje manual, cada número es la velocidad de un motor, y su signo da el sentido: positivo avanza, negativo retrocede. Con eso se arma cualquier movimiento. `M:200,200` avanza recto; `M:-200,-200` retrocede; `M:200,100` avanza curvando, porque la rueda izquierda gira más rápido que la derecha; `M:-150,150` gira en el lugar, y `M:0,0` detiene el robot. Es la tracción diferencial completa en dos números.

El Código 6.1 retoma las funciones del Capítulo 5 y agrega tres piezas. `procesarDatos()` revisa cómo empieza el mensaje y, si es manual, separa los dos números con `indexOf()` y `substring()`. `controlManualApp()` da a cada motor su sentido y su velocidad según el signo y el valor absoluto del número. Y `loop()` solo pregunta si llegó una línea nueva.

```
// Control manual del robot por Bluetooth con la app Guia_Robotica (Capítulo 6).
// El HC-05 va al puerto serial del NodeMCU (RX y TX): desconéctalo antes de
// subir el programa. La app manda una línea de texto como  M:150,-150  :
// la velocidad de cada motor, de -255 (atrás a fondo) a 255 (adelante a fondo).

// Motores (módulo L298N, ver Tabla 4.2, Capítulo 4)
const int ENA = D8, IN1 = D7, IN2 = D6;  // Motor izquierdo
const int ENB = D3, IN3 = D5, IN4 = D4;  // Motor derecho

void detener() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

// Cada motor recibe un número con signo: el signo da el sentido y el valor
// absoluto, la velocidad.
void controlManualApp(int mIzq, int mDer) {
  if (mIzq >= 0) {
    digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);  // hacia adelante
  } else {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);   // en reversa
  }
  analogWrite(ENA, abs(mIzq));

  if (mDer >= 0) {
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);   // hacia adelante
  } else {
    digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);  // en reversa
  }
  analogWrite(ENB, abs(mDer));
}

// Lee un mensaje  M:izquierdo,derecho  y mueve los motores.
// Cualquier otro mensaje (por ejemplo, el de «Enviar PID») se ignora.
void procesarDatos(String d) {
  if (!d.startsWith("M:")) return;

  int coma = d.indexOf(",");
  if (coma == -1) return;

  int motorIzq = d.substring(2, coma).toInt();
  int motorDer = d.substring(coma + 1).toInt();
  controlManualApp(constrain(motorIzq, -255, 255), constrain(motorDer, -255, 255));
}

void setup() {
  Serial.begin(9600);  // velocidad de fábrica del HC-05
  Serial.setTimeout(10);
  analogWriteRange(255);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
  detener();
}

void loop() {
  if (Serial.available() > 0) {
    String data = Serial.readStringUntil('\n');
    data.trim();
```

```
64      if (data.length() > 0) {
65        procesarDatos(data);
66      }
67    }
68  }
```

Código 6.1. Recepción de mensajes por Bluetooth y control manual del robot.

NOTA

Este programa solo maneja el robot a mano: el mensaje de «Enviar PID», que empieza con `KP:`, lo ignora. El Capítulo 7 le da sentido.

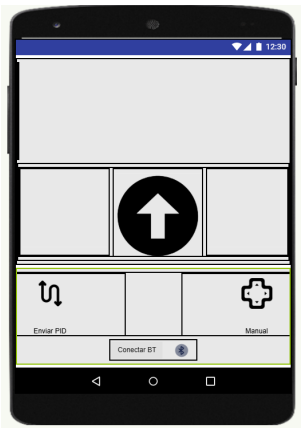
6.4 La app de control en MIT App Inventor

La app del robot, `Guia_Robotica`, está en el repositorio del libro (Anexo C) en dos formatos: el proyecto `.aia`, que se abre y se edita en MIT App Inventor, y el instalador `.apk`, listo para el celular. Necesitas un celular **Android**: las apps de App Inventor con Bluetooth clásico no funcionan en iPhone.

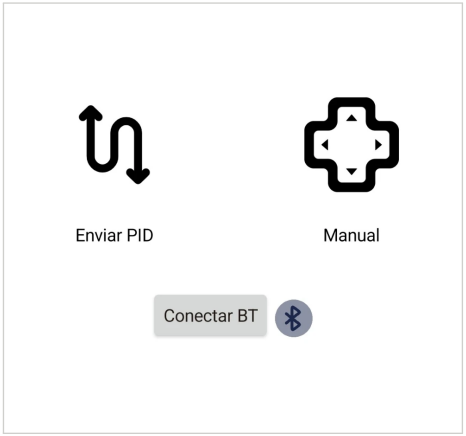
MIT App Inventor tiene dos vistas: el **Diseñador**, donde se acomodan botones y otros elementos visuales, y **Bloques**, donde se define qué hace cada uno. Para abrir la app, entra a App Inventor y usa *Proyectos* → *Importar proyecto* (*.aia*) *desde mi ordenador*. Recórrela así:

1. **Las pantallas.** Todo vive en una sola pantalla con tres grupos (*arrangements*) que se muestran o se esconden: el menú principal, la pantalla «Manual» (el control de movimiento) y la pantalla «Enviar PID», con tres deslizadores (KP, KI y KD) y el botón «Enviar». La flecha de la esquina vuelve al menú. El componente `BluetoothClient` no es visible: es el que habla con el HC-05.
2. **La conexión.** El selector de lista «Conectar BT» (`ListPicker`) carga, antes de abrirse (`BeforePicking`), los dispositivos emparejados (`BluetoothClient1.AddressesAndNames`); al elegir uno (`AfterPicking`), llama a `BluetoothClient1.Connect` y avisa «Conectado!».
3. **El control manual.** La pantalla «Manual» tiene cuatro botones. Al presionar uno (evento `TouchDown`), la app guarda los valores de los dos motores: adelante, 150 y 150; atrás, −150 y −150; izquierda, 0 y 150; derecha, 150 y 0. Al soltarlo (`TouchUp`), los valores vuelven a 0 y 0. Un temporizador (`Clock`) manda con `BluetoothClient1.EnviarTexto` el mensaje `M:` con los valores guardados, cada 0.2 segundos, mientras estás en esa pantalla. Por eso el robot se mueve solo mientras mantienes el dedo encima y se detiene al soltar.
4. **Las ganancias del PID.** En «Enviar PID», cada deslizador fija una ganancia y su etiqueta muestra el valor (por ejemplo, «KP:500»); KP va de 0 a 1000, y KI y KD, de 0 a 100. El botón «Enviar» (evento `Click`) llama a `BluetoothClient1.EnviarTexto` con un solo texto: los valores de los tres deslizadores unidos con sus nombres y un salto de línea. Los botones «+» y «–» mueven el deslizador de KP en pequeños pasos. Para volver a manejar a mano, usa la flecha de la esquina y entra a «Manual». El Capítulo 7 explica qué son esas tres ganancias.
5. **La protección.** Antes de enviar, cada bloque revisa si hay conexión; si no la hay, muestra el aviso «Bluetooth no conectado» en vez de fallar en silencio.

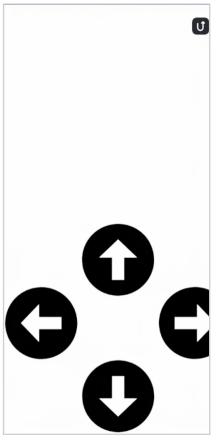
La Figura 6.1 muestra el Diseñador, la app ya corriendo en el celular (el menú principal con «Conectar BT» y la pantalla «Manual») y los Bloques.



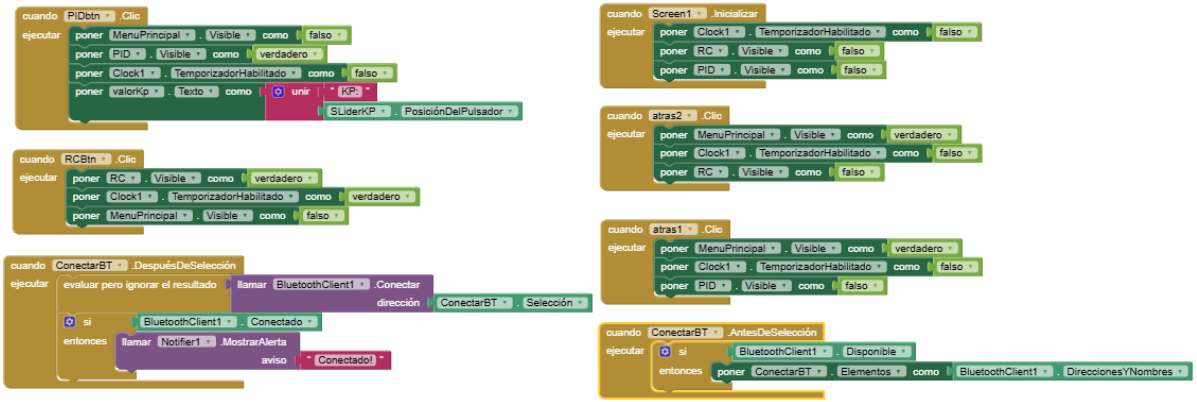
(a) Diseñador



(b) Menú principal



(c) Control manual



(d) Bloques: conexión y menús

Figura 6.1. La app de control Guia_Robotica en MIT App Inventor: el Diseñador, el menú principal y el control manual en el celular, y los bloques de navegación y conexión.

CONSEJO PRÁCTICO

Para probar cambios en la app, usa la app «MIT AI2 Companion» en tu celular antes de exportar un `.apk` nuevo. Así detectas errores de bloques sin esperar a tener el robot encendido al lado.

6.5 Manos a la obra: empareja y controla tu robot

MANOS A LA OBRA

1. Con el HC-05 desconectado, sube el Código 6.1 al NodeMCU. Después conecta el HC-05 según la Tabla 6.1.
2. Enciende el robot: el LED del HC-05 parpadea rápido mientras espera conexión. Búscalo en el Bluetooth de tu celular como `HC-05` y empareja; el código por defecto suele ser `1234` o `0000`.
3. Instala la app: el `Guia_Robotica.apk` del repositorio (el celular te pedirá permitir apps de origen desconocido), o el proyecto `.aia` a través del Companion.
4. Abre la app, toca «Conectar BT», elige el HC-05 y entra al «Manual». Al conectarse, el

LED del módulo cambia a un parpadeo lento.
5. Prueba los cuatro botones: adelante, atrás, izquierda y derecha. Al soltar el botón, el robot debe detenerse.
6. Si el robot no responde, revisa en Bloques que el mensaje empiece con `M:` y lleve dos números separados por una coma, y en el programa la función `procesarDatos()`: es el error más común en esta etapa.
7. **Hazla tuya.** Cambia el `255` de `constrain()` en `procesarDatos()` por un valor menor, como 150, y observa cómo cambia la velocidad máxima. Después, en App Inventor, agrega un botón que mande `M:200,-200`, para que el robot gire en el lugar.

ANTES DE SEGUIR, VERIFICA

- [ ] El celular encuentra y empareja el HC-05.
- [ ] La app se conecta al módulo sin errores.
- [ ] Los cuatro botones de la pantalla «Manual» mueven el robot mientras los presionas y lo detienen al soltarlos.
- [ ] Sabes qué mensaje manda la pantalla «Manual» y cuál manda «Enviar PID», que usarás en el Capítulo 7.
- [ ] Sabes que el HC-05 se desconecta antes de cada carga de programa.

RESUMEN DEL CAPÍTULO

- Un protocolo es un acuerdo sobre qué significa cada dato: el de tu robot tiene dos mensajes de texto, uno para manejar a mano y otro para el PID.
- El HC-05 va al puerto serial de hardware del NodeMCU: se lee con `Serial`, sin librerías, pero hay que desconectarlo para cargar programas.
- NodeMCU y HC-05 trabajan a 3.3 V: se conectan directo, sin divisor de tensión, con las líneas de datos cruzadas.
- MIT App Inventor convierte lo que haces en la pantalla en mensajes Bluetooth que tu NodeMCU ya sabe interpretar; con el proyecto `.aia` puedes cambiar la app a tu gusto.

7 Seguidor de línea y control PID

EN ESTE CAPÍTULO APRENDERÁS

- Entender cómo un robot decide qué hacer a partir de lo que mide.
- Convertir lo que ven los dos sensores en un número: el error.
- Saber, sin matemática avanzada, qué hacen las partes P, I y D de un controlador PID.
- Cargar el programa del seguidor, ajustar sus ganancias y alternar entre manejo manual y automático desde el celular.

Hasta aquí tu robot hace lo que tú le ordenas: avanza, gira, se detiene. En este capítulo decide solo. Va a seguir una cinta negra pegada en el piso, y para lograrlo vas a conocer una de las ideas más usadas de la ingeniería: el control PID.

7.1 Seguir una línea es un problema de control

Piensa en cómo caminas por un pasillo. No calculas nada: miras dónde estás, notas que te acercas a la pared y te corriges, y vuelves a mirar. Un robot seguidor de línea hace exactamente eso, muchas veces por segundo, en cuatro pasos que se repiten sin parar (Figura 7.1).

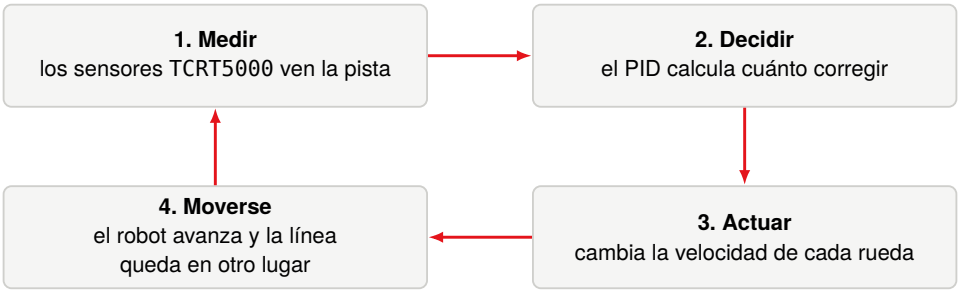
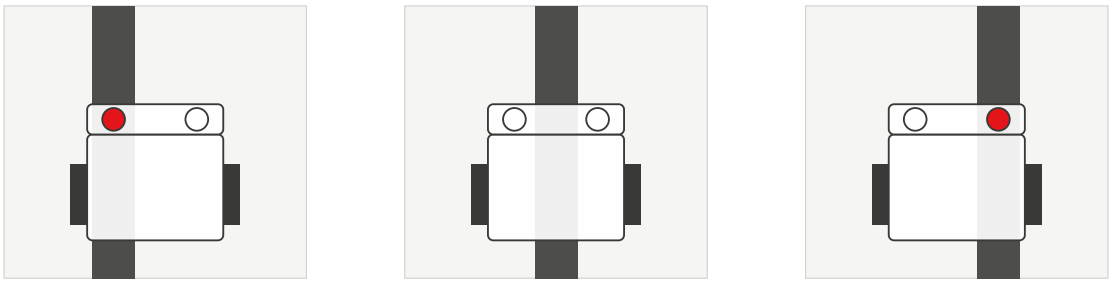


Figura 7.1. El ciclo de control del seguidor de línea. Como el resultado de moverse vuelve a medirse, se llama control con retroalimentación.

Que el resultado vuelva a medirse es lo que lo hace potente: si te pasas de largo, el siguiente ciclo lo nota y lo corrige. El **PID** (Proporcional-Integral-Derivativo) es la receta más común para el segundo paso: la que decide cuánto corregir.

7.2 De los sensores a un número: el error

Tus dos sensores van en el frente del chasis, separados un poco más que el ancho de la cinta. Con el robot bien centrado, la cinta pasa justo entre ellos y los dos ven la superficie clara. Si el robot se desvía, la cinta queda debajo de uno de los dos. La Figura 7.2 muestra los tres casos.



error $+1$

la cinta quedó bajo el sensor izquierdo: gira a la izquierda

error 0

la cinta pasa entre los sensores: avanza recto

error -1

la cinta quedó bajo el sensor derecho: gira a la derecha

Figura 7.2. Lo que ven los dos sensores según dónde quede la línea (vista desde arriba). Un sensor sobre la cinta se muestra en rojo.

El **error** es el número que resume esa situación. Vale 0 si los dos sensores ven la superficie clara (el robot está centrado), $+1$ si la cinta quedó bajo el sensor izquierdo y -1 si quedó bajo el derecho. El signo dice hacia dónde hay que corregir: $+1$ pide girar a la izquierda y -1, a la derecha.

Queda un caso especial: los dos sensores indican «cinta» a la vez, o no reciben luz (por ejemplo, si levantas el robot). El programa lo resuelve con memoria, que se actualiza solo mientras el robot está sobre la pista. Si venía centrado, se detiene por seguridad. Si el último error no era cero, deja de aplicar el PID y gira sobre su propio eje, hacia el lado que indicaba ese último error, hasta que los sensores vuelven a ver la pista.

NOTA

Con dos sensores de sí o no, el error solo vale -1, 0 o $+1$. Un ángulo de 90° o una línea que cruza ponen los dos sensores sobre la cinta a la vez, y el robot entra en ese modo de búsqueda o frena. Por eso la pista debe tener curvas amplias, y el robot debe ir a una velocidad moderada. Si se sale por completo, vuelve al menú con la flecha de la app y entra a «Manual»: el robot deja de seguir la línea y vuelve a obedecerte.

7.3 Qué es un PID, con palabras simples

Imagina que andas en bicicleta siguiendo una línea pintada en el piso. Sin pensarlo, haces tres cosas con el manubrio:

- Giras más fuerte cuanto más lejos estás de la línea.
- Cuando ya vienes volviendo hacia ella, enderezas el manubrio antes de llegar, para no cruzarla y pasarte al otro lado.
- Si un viento lateral te empuja siempre hacia el mismo lado, notas que llevas rato desviado y giras cada vez un poco más.

Esas tres reacciones son las tres partes del PID. El controlador calcula una **corrección** sumando las tres, y cada una trae su propia «perilla» (una ganancia, que es un número que tú ajustas) (Tabla 7.1).

En una sola línea, el cálculo es este:

Tabla 7.1. Las tres partes de un controlador PID.

Parte	Pregunta que responde	Qué hace	Si te pasas
P, proporcional (K_p)	¿Qué tan desviado estoy ahora?	Corrige en proporción al error: más desvío, más giro.	El robot zigzaguea y pasa de un lado a otro de la línea.
I, integral (K_i)	¿Cuánto tiempo llevo desviado?	Suma el error con el tiempo: si el desvío persiste, empuja cada vez más.	Se pasa de largo y oscila despacio; por eso se empieza en cero.
D, derivativa (K_d)	¿Qué tan rápido cambia el desvío?	Mira el cambio del error y frena la corrección cuando el robot ya vuelve a la línea.	Reacciona a todo y el robot vibra.

$$\text{corrección} = K_p \cdot e \;+\; K_i \cdot (\text{suma de los errores}) \;+\; K_d \cdot (e - e_{\text{anterior}})$$

donde e es el error de ahora y e_{anterior} el del ciclo previo. Un PID de manual multiplicaría la suma por el tiempo entre cálculos y dividiría el cambio por ese mismo tiempo. Aquí no hace falta: el programa repite el cálculo una y otra vez a un ritmo parecido, así que ese tiempo queda incluido dentro de K_i y K_d. La corrección se reparte entre las ruedas: a la izquierda se le resta y a la derecha se le suma. Un ejemplo con la velocidad base de 200 y $K_p = 400$: si la cinta quedó bajo el sensor izquierdo ($e = +1$), la corrección es $+400$; la rueda izquierda iría a $200 - 400 = -200$, pero el programa no deja valores negativos y la deja en 0; la derecha va a $200 + 400 = 600$, que el programa recorta al máximo que admite el motor. El robot gira a la izquierda.

NOTA

Con sensores de sí o no, el error solo toma tres valores (-1, 0 y $+1$). Por eso, en este robot, la parte P hace casi todo el trabajo, y I y D se notan poco. Se lucen con sensores que miden *qué tan lejos* está la línea (sensores analógicos o una barra con varios), donde el error toma muchos valores. Aun así, conviene aprender las tres partes juntas: son las mismas que usan un dron, una calefacción o el control de crucero de un auto.

7.4 El programa del seguidor

El Código 7.1 es el programa definitivo del robot. Reúne todo lo que construiste y le suma el piloto automático. Se organiza en cinco bloques:

- `calcularError()` lee los dos sensores y devuelve el error: -1, 0 o $+1$. Si los dos indican cinta, mira `previous_error` (el último error sobre la pista) y marca el caso especial: 100 si venía centrado (frenar) o ± 10 (buscar la línea).
- `calcularPID()` aplica la fórmula: `P` es el error, `I` la suma de los errores y `D` el cambio del error. En los casos especiales no calcula nada.
- `aplicarMotores()` reparte la corrección: la rueda izquierda recibe la velocidad base menos la corrección y la derecha, más la corrección. En los casos especiales, frena o gira sobre su eje.

- `procesarDatos()` entiende los dos mensajes de la app: el de «Enviar PID» (empieza con `KP:`) fija las tres ganancias y activa el modo automático, y el manual (empieza con `M:`) lo desactiva.
- `controlManualApp()` mueve cada motor con el número que llega, igual que en el Capítulo 6.

Las ganancias y la velocidad base están juntas, al comienzo del programa: $K_p = 400$, $K_i = 0$, $K_d = 0$ y `velocidad_base` $= 200$. Con esos valores, el robot del autor recorre la pista completa. Son un punto de partida, no la respuesta: cada robot tiene motores, peso y pilas distintos, y encontrar los suyos es el ejercicio de la próxima sección. La app manda las ganancias tal cual: el deslizador de KP va de 0 a 1000, y los de KI y KD, de 0 a 100.

NOTA

El programa está escrito para el robot del libro: el sensor izquierdo en `D1` y el derecho en `D0`, `LOW` sobre la superficie clara y `HIGH` sobre la cinta. Si tu robot corrige hacia el lado contrario, es decir, se aleja de la cinta en vez de volver a ella, intercambia los cables de señal de los dos sensores. Si tu módulo entrega los niveles al revés, cambia `LOW` por `HIGH` (y al revés) en `calcularError()`. Y si alguna rueda gira en sentido contrario, intercambia los cables de ese motor.

La Figura 7.3 muestra la pantalla «Enviar PID» y los bloques que arman el mensaje.



(**a**) Pantalla «Enviar PID» (**b**) Bloques: etiquetas, botones «+» y «–», y «Enviar»

Figura 7.3. Cómo manda la app las ganancias del PID por Bluetooth.

```
// Seguidor de línea con control PID y control manual por Bluetooth (Capítulo 7).
// El HC-05 va al puerto serial del NodeMCU (RX y TX): desconéctalo antes de
// subir el programa. La app manda dos tipos de mensaje, cada uno en una línea:
//   KP:400,KI:0,KD:0   ganancias del PID: inicia el modo automático
//   M:150,-150         velocidad de cada motor (-255 a 255): modo manual

// --- PINES DE MOTORES (módulo L298N, ver Tabla 4.2, Capítulo 4) ---
int in1 = D7;
int in2 = D6;
int in3 = D5;
int in4 = D4;
int ENA = D8;   // motor izquierdo
int ENB = D3;   // motor derecho

// --- PINES DE SENSORES (ver Tabla 4.3, Capítulo 4) ---
// Con los módulos del libro, cada sensor entrega LOW sobre la superficie
// clara y HIGH sobre la cinta negra (o si no recibe luz, por ejemplo al
// levantar el robot).
int irLeft = D1;
int irRight = D0;

```

```
22 // --- VARIABLES DEL PID ---
23 float Kp = 400.0;
24 float Ki = 0.0;
25 float Kd = 0.0;
26 int velocidad_base = 200;
27
28 // Variables matemáticas internas
29 float P = 0, I = 0, D = 0, PID_value = 0;
30 float error = 0, previous_error = 0;
31
32 // --- VARIABLES BLUETOOTH Y SISTEMA ---
33 bool modoAuto = true; // true = Seguidor de Línea, false = Control Manual
34
35 void setup() {
36   Serial.begin(9600);
37   Serial.setTimeout(10);
38
39   pinMode(in1, OUTPUT);
40   pinMode(in2, OUTPUT);
41   pinMode(in3, OUTPUT);
42   pinMode(in4, OUTPUT);
43   pinMode(ENA, OUTPUT);
44   pinMode(ENB, OUTPUT);
45
46   // Parche de seguridad
47   digitalWrite(in1, LOW);
48   digitalWrite(in2, LOW);
49   digitalWrite(in3, LOW);
50   digitalWrite(in4, LOW);
51   analogWrite(ENA, 0);
52   analogWrite(ENB, 0);
53
54   pinMode(irLeft, INPUT);
55   pinMode(irRight, INPUT);
56
57   Serial.println("Sistema encendido. Esperando datos...");
58   delay(3000);
59 }
60
61 void loop() {
62   // 1. LEER BLUETOOTH
63   if (Serial.available() > 0) {
64     String data = Serial.readStringUntil('\n');
65     data.trim();
66
67     if (data.length() > 0) {
68       procesarDatos(data);
69     }
70   }
71
72   // 2. EJECUTAR NAVEGACIÓN CONTINUA
73   if (modoAuto) {
74     calcularError();
75     calcularPID();
76     aplicarMotores();
77   }
78 }
79
80 // ============================================================
81 // BLOQUE 1: CÁLCULO DEL ERROR (con identificación de pérdida)
82 // ============================================================
83 void calcularError() {
84   int leftVal   = digitalRead(irLeft);
```

```
  int rightVal  = digitalRead(irRight);

  if (leftVal == LOW && rightVal == HIGH) {
    error = -1;   // la cinta quedó bajo el sensor derecho
  }
  else if (leftVal == HIGH && rightVal == LOW) {
    error = 1;    // la cinta quedó bajo el sensor izquierdo
  }
  else if (leftVal == LOW && rightVal == LOW) {
    error = 0;    // ambos sobre la superficie clara: la cinta pasa entre ellos
  }
  else if (leftVal == HIGH && rightVal == HIGH) {
    // Ambos sensores sin luz de vuelta: se mira la memoria
    if (previous_error == 0) {
      error = 100; // Freno de seguridad si se levanta en recta
    }
    else if (previous_error < 0) {
      error = -10; // Perdió la cinta con el último error negativo
    }
    else if (previous_error > 0) {
      error = 10;  // Perdió la cinta con el último error positivo
    }
  }
}

// ============================================================
// BLOQUE 2: MATEMÁTICA PID
// ============================================================
void calcularPID() {
  // Ignorar matemática si está perdido haciendo el trompo o frenado
  if (abs(error) == 10 || error == 100) return;

  P = error;
  I = I + error;
  D = error - previous_error;

  PID_value = (Kp * P) + (Ki * I) + (Kd * D);
}

// ============================================================
// BLOQUE 3: POTENCIA Y DIRECCIÓN (con giro de tanque)
// ============================================================
void aplicarMotores() {

  // --- RECUPERACIÓN: BÚSQUEDA SOBRE SU PROPIO EJE ---
  if (abs(error) == 10) {
    int searchSpeed = 150; // Velocidad de giro. Ajustar si gira muy brusco.

    if (error == -10) {
      // Gira a la izquierda (rueda izquierda atrás, rueda derecha adelante)
      digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
      digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
      analogWrite(ENA, searchSpeed);
      analogWrite(ENB, searchSpeed);
    }
    else if (error == 10) {
      // Gira a la derecha (rueda izquierda adelante, rueda derecha atrás)
      digitalWrite(in1, LOW); digitalWrite(in2, HIGH);
      digitalWrite(in3, LOW); digitalWrite(in4, HIGH);
      analogWrite(ENA, searchSpeed);
      analogWrite(ENB, searchSpeed);
    }
    return; // Salir de la función hasta que vuelva a ver la línea
```

```
148     }
149
150     // --- FRENO TOTAL (si error es 100) ---
151     if (error == 100) {
152       digitalWrite(in1, LOW); digitalWrite(in2, LOW);
153       digitalWrite(in3, LOW); digitalWrite(in4, LOW);
154       analogWrite(ENA, 0); analogWrite(ENB, 0);
155       return;
156     }
157
158     // --- AVANCE NORMAL CON PID ---
159     int speedLeft = velocidad_base - PID_value;
160     int speedRight = velocidad_base + PID_value;
161
162     speedLeft = constrain(speedLeft, 0, 1023);
163     speedRight = constrain(speedRight, 0, 1023);
164
165     // Dirección hacia adelante
166     digitalWrite(in1, LOW);
167     digitalWrite(in2, HIGH);
168     digitalWrite(in3, HIGH);
169     digitalWrite(in4, LOW);
170
171     analogWrite(ENA, speedLeft);
172     analogWrite(ENB, speedRight);
173
174     // Solo guardamos memoria si estamos pisando la pista
175     if (error == -1 || error == 1 || error == 0) {
176       previous_error = error;
177     }
178   }
179
180   // ============================================================
181   // BLOQUE 4: COMUNICACIÓN CON LA APP
182   // ============================================================
183   void procesarDatos(String d) {
184     if (d.startsWith("KP:")) {
185       modoAuto = true;
186
187       int iKP = d.indexOf("KP:");
188       int iKI = d.indexOf(",KI:");
189       int iKD = d.indexOf(",KD:");
190
191       if (iKP != -1 && iKI != -1 && iKD != -1) {
192         Kp = d.substring(iKP + 3, iKI).toFloat();
193         Ki = d.substring(iKI + 4, iKD).toFloat();
194         Kd = d.substring(iKD + 4).toFloat();
195
196         Serial.print("NUEVO PID -> P:"); Serial.print(Kp);
197         Serial.print(" I:"); Serial.print(Ki);
198         Serial.print(" D:"); Serial.println(Kd);
199
200         I = 0;
201       }
202     }
203     else if (d.startsWith("M:")) {
204       modoAuto = false;
205
206       int coma = d.indexOf(",");
207       if (coma != -1) {
208         int motorIzq = d.substring(2, coma).toInt();
209         int motorDer = d.substring(coma + 1).toInt();
210
```

```
211         controlManualApp(motorIzq, motorDer);
212       }
213     }
214   }
215
216   // ============================================================
217   // BLOQUE 5: CONTROL MANUAL
218   // ============================================================
219   void controlManualApp(int mIzq, int mDer) {
220     if (mIzq >= 0) {
221       digitalWrite(in1, LOW); digitalWrite(in2, HIGH);
222       analogWrite(ENA, mIzq);
223     } else {
224       digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
225       analogWrite(ENA, abs(mIzq));
226     }
227
228     if (mDer >= 0) {
229       digitalWrite(in3, HIGH); digitalWrite(in4, LOW); // Hacia adelante
230       analogWrite(ENB, mDer);
231     } else {
232       digitalWrite(in3, LOW); digitalWrite(in4, HIGH); // Reversa
233       analogWrite(ENB, abs(mDer));
234     }
235   }
```

Código 7.1. Seguidor de línea con control PID y modo manual por Bluetooth.

7.5 Manos a la obra: pista y sintonía

MANOS A LA OBRA

1. **Arma la pista.** Pega la cinta negra sobre una superficie clara y sin brillo, formando un óvalo o un circuito de curvas amplias (de unos 30 cm de radio o más) y sin ángulos de 90°. Si la cinta es muy brillante y refleja luz, el sensor puede verla como blanca: usa cinta mate o un papel negro opaco.
2. **Carga el programa.** Descarga el código definitivo (extttCodigoDefinitivo.ino) del repositorio (Anexo C) o escríbelo desde el Código 7.1. Con el HC-05 desconectado, súbelo al NodeMCU. Revisa que tus sensores entreguen `LOW` sobre la superficie clara y `HIGH` sobre la cinta, como en la calibración del Capítulo 5. Vuelve a conectar el HC-05.
3. **Prueba el modo manual** con la pantalla «Manual» de la app: debe comportarse como en el Capítulo 6.
4. **Pon el robot sobre la pista**, con la cinta entre los dos sensores. En la app, entra a «Enviar PID», deja KP en 400 y lleva los deslizadores de KI y KD a 0. Toca «Enviar»: el robot empieza a seguir la línea.
5. **Ajusta K_p primero.** Con KI y KD en 0, parte de 400, el valor del autor. Si el robot se sale en las curvas, sube el deslizador; si zigzaguea mucho en las rectas, bájalo. Los botones «+» y «–» afinan el valor. Cada cambio se prueba con «Enviar», sin volver a subir el programa. Cambia un solo número por vez y anótalo.
6. **Prueba K_d.** Con el KP que mejor te funcionó, sube el deslizador de KD de a poco (de 10 en 10). Observa si el movimiento se suaviza. Con sensores de sí o no puede notarse poco: es una conclusión válida.
7. **Deja K_i para el final.** Mantenlo en 0. El programa suma el error en cada vuelta, muchas

veces por segundo, así que K_i crece rápido: si lo pruebas, usa valores muy pequeños (1 o menos) y compara.

8. **Sube la velocidad.** Aumenta `velocidad_base` en el programa de a poco, vuelve a subirlo y reajusta KP. Mide cuánto tarda tu robot en dar una vuelta a la pista y trata de bajar ese tiempo.

Tabla 7.2. Síntomas del seguidor y qué probar.

Síntoma	Qué probar
Se sale en las curvas	Sube `KP` o baja `velocidad_base`
Zigzaguea mucho en las rectas	Baja `KP`, sube un poco `KD` o baja la velocidad
Se desvía siempre hacia el mismo lado	Revisa que ambos motores giren igual (cableado, pilas); después prueba un `KI` pequeño
Se corrige hacia el lado contrario, o gira en el lugar	Revisa los niveles `LOW/HIGH`, la calibración (Capítulo 5) y que los cables de los sensores no estén intercambiados
Se detiene de golpe en una recta	Los dos sensores indicaron cinta a la vez: revisa que no lo levantes, que la pista no tenga una raya que cruza y la altura de los sensores
Se comporta distinto con pilas nuevas y gastadas	Las pilas cambian la velocidad real de los motores: prueba siempre con un juego nuevo

ANTES DE SEGUIR, VERIFICA

- ☐ Puedes explicar con tus palabras qué es el error y qué hace cada parte del PID.
- ☐ El robot sigue la pista completa en modo automático, sin salirse.
- ☐ La flecha de la app y la pantalla «Manual» te devuelven el control del robot en cualquier momento.
- ☐ Anotaste los valores de `KP`, `KI`, `KD` y `velocidad_base` con los que tu robot rinde mejor.

RESUMEN DEL CAPÍTULO

- Seguir una línea es un ciclo que se repite: medir, decidir, actuar y volver a medir. Esa retroalimentación es la base del control automático.
- Dos sensores bastan para resumir la situación en un número, el error: -1, 0 o $+1$, con un modo de búsqueda cuando los dos indican cinta.
- El PID suma tres reacciones: P corrige según qué tan desviado estás, I según cuánto tiempo llevas desviado y D según qué tan rápido cambia el desvío. Cada una tiene su ganancia.
- Con sensores de sí o no, P hace casi todo el trabajo. Las ganancias se encuentran probando, de a una por vez, y anotando los resultados.
- El mismo programa maneja el robot a mano o en automático: el mensaje de «Enviar PID» fija las ganancias y pone en marcha el seguidor, y el mensaje manual devuelve el control.

8 Epílogo

Felicidades. Si llegaste hasta aquí con el robot recorriendo la pista por sí solo y respondiendo a tu celular cuando tomas el control, no armaste solo un vehículo con motores: pasaste por las cuatro capas que el Capítulo 0 prometía —mecánica, eléctrica, electrónica e informática— y las hiciste convivir en una sola máquina.

ANTES DE SEGUIR, VERIFICA

- [ ] El chasis diseñado en OnShape está impreso y todos los componentes encajan en él, con los sensores al frente a la altura correcta.
- [ ] Los motores, el módulo L298N, el NodeMCU con su shield y los sensores `TCRT5000` están cableados y alimentados sin GND flotante.
- [ ] El código de la rutina de prueba corrió sin errores y los dos sensores distinguen la cinta negra del fondo.
- [ ] La app de MIT App Inventor maneja el robot por Bluetooth: avanza, retrocede, gira y curva, e inicia y detiene el seguidor de línea.
- [ ] El robot completa la pista en modo automático con las ganancias del PID que tú mismo encontraste.

8.1 Retos para tu pista

No necesitas más que cinta, una superficie clara y un cronómetro. Prueba estos retos, del más simple al más difícil:

- **Vuelta rápida.** Mide cuánto tarda tu robot en dar una vuelta completa y baja ese tiempo ajustando `KP` y `VELOCIDAD_BASE`.
- **Pista nueva.** Diseña un circuito con curvas más cerradas y averigua hasta dónde llega tu robot. ¿Qué cambia en las ganancias?
- **Carrera.** Si tienes compañeros con su propio robot, corran el mismo circuito: gana quien dé más vueltas en dos minutos sin salirse, y nadie puede tocar su robot hasta que termine.

8.2 Lo que aprendiste

Lo que realmente construiste es transferible: diseñar una pieza para que cumpla una función, entender por qué un motor necesita un puente H antes de recibir corriente, hacer que un programa decida a partir de lo que mide, y definir un protocolo simple para que dos dispositivos se entiendan. Esas cuatro habilidades sirven para el próximo robot, no solo para este.

8.3 Hacia dónde seguir

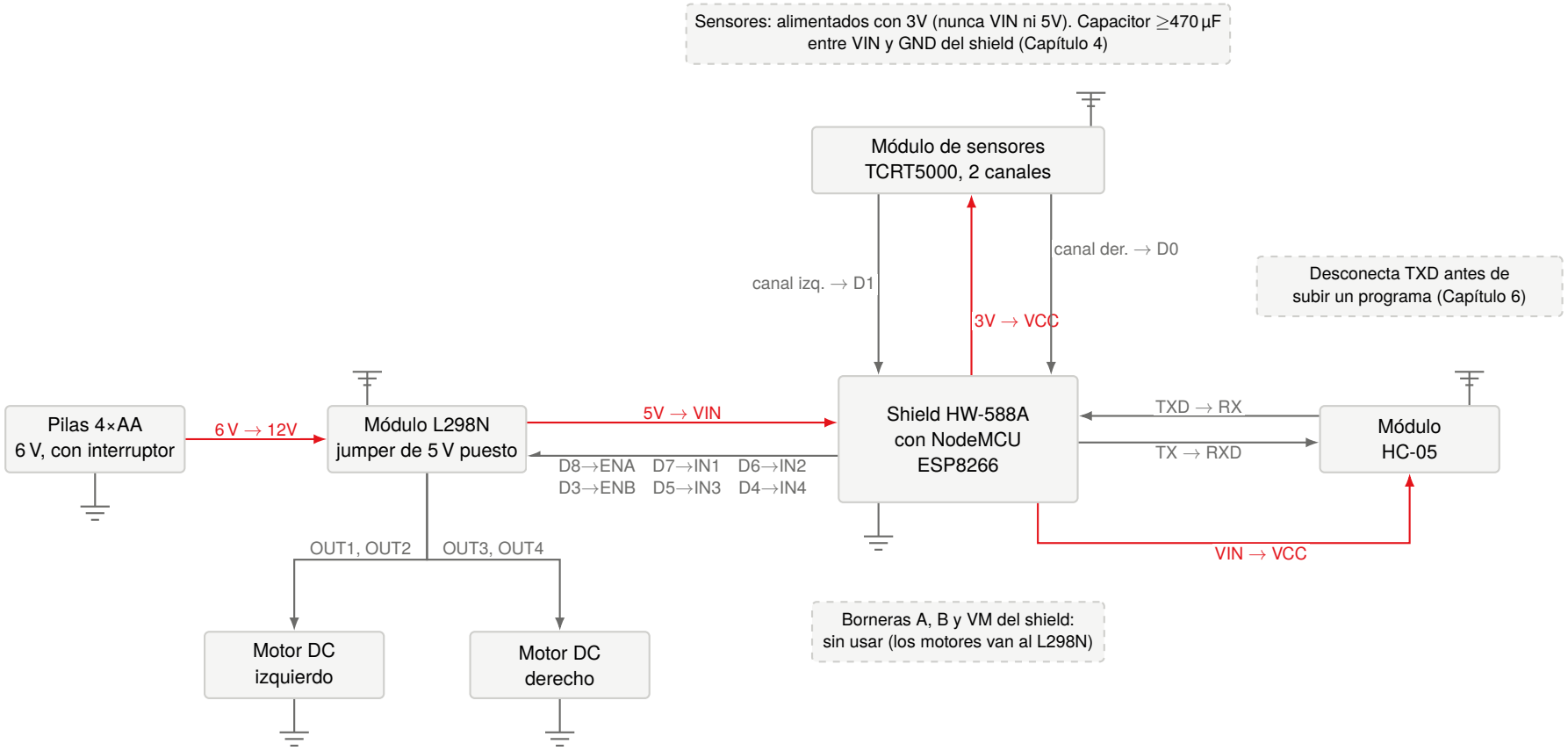
Con los mismos componentes —o agregando uno o dos sensores— puedes construir:

- Un **seguidor más fino**, con una barra de cinco sensores o sensores analógicos: el error toma más valores y las partes I y D del PID se notan de verdad.
- Una **memoria de ganancias**: guardar en el NodeMCU los valores que enviaste desde la app, para que no se pierdan al apagar el robot.
- Un **evasor de obstáculos**, con un sensor ultrasónico.
- Un **brazo robótico**, agregando servomotores.
- Control por **WiFi** en vez de Bluetooth: tu NodeMCU ya lo trae integrado, sin ningún módulo extra.
- Un robot con **visión**, agregando una cámara y un poco de procesamiento de imágenes.

Cada uno de esos proyectos parte del mismo lugar donde tú ya estás parado.

Gracias por construir hasta el final. Ahora te toca seguir la línea.

A Diagrama de conexiones



Todos los símbolos de tierra (⊥) están unidos entre sí: GND común obligatorio (Capítulo 4).

B Guía de resolución de problemas

Todas las fallas de esta guía ya aparecieron, una por una, en su capítulo correspondiente. Aquí están juntas, por síntoma, para revisar rápido sin tener que recorrer todo el libro.

Tabla B.1. Impresión 3D (Capítulo 3).

Síntoma	**Causa probable**	**Solución**
Las esquinas se levantan de la cama (*warping*)	Mala adherencia o enfriamiento desigual	Usa *brim*, limpia la cama, evita corrientes de aire
Hilos finos entre piezas (*stringing*)	Temperatura muy alta o falta de retracción	Baja 5–10 °C la boquilla, activa retracción
La pieza se despega a mitad de impresión	Cama sucia, sin nivelar, o primera capa muy rápida	Limpia con alcohol isopropílico, nivela, reduce velocidad de la primera capa

Tabla B.2. Electrónica y alimentación (Capítulo 4).

Síntoma	Causa probable	Solución
El NodeMCU se reinicia al arrancar los motores	La tensión de `VIN` cae: pilas gastadas, o el regulador del L298N no alcanza cuando los motores piden su pico de corriente	Mide `VIN` mientras los motores arrancan. Cambia las pilas por alcalinas nuevas y conserva el capacitor de $\geq$470 µF entre `VIN` y `GND`
El NodeMCU no arranca, o su LED queda fijo, con todo conectado	`D3`, `D4` o `D8` quedan en un nivel que impide el arranque	Desconecta esos tres cables, enciende, y vuelve a conectarlos uno por uno para encontrar el culpable
El robot se comporta de forma errática o no responde	Falta GND común entre alguna etapa	Verifica continuidad de GND entre pilas, L298N, shield y sensores
Los motores giran siempre a fondo, aunque cambies la velocidad	Siguen puestos los jumpers de `ENA` y `ENB` del L298N	Retíralos y conecta ahí los cables de `D8` y `D3`
Un motor gira al revés de lo esperado	Cables del motor invertidos en la bornera	Intercambia los dos cables de ese motor en el L298N
El robot pierde fuerza después de un rato de uso	Pilas alcalinas descargándose bajo carga	Cambia las pilas. Cuatro recargables NiMH entregan solo 4.8 V: no alcanzan para este montaje
El LED de un sensor no cambia entre blanco y cinta	El umbral del potenciómetro no calza con la altura, o el sensor está muy alto o muy bajo	Acerca o aleja el sensor a unos 5–10 mm del piso y gira el potenciómetro hasta que el LED cambie
Un pin de entrada del NodeMCU deja de responder	Los sensores están alimentados con 5 V (`VIN`) en vez de 3.3 V, y sus salidas de 5 V dañaron el pin	Aliméntalos desde el pin `3V` del NodeMCU y cambia el sensor a otro pin libre si el daño es permanente

Tabla B.3. Programación (Capítulo 5).

Síntoma	Causa probable	Solución
El Arduino IDE no muestra ningún puerto	Falta el controlador `CP2102`, o el cable USB solo carga	Instala el controlador de Silicon Labs y prueba con otro cable
La carga falla con un error de conexión	El HC-05 sigue conectado a `RX`/`TX`, o no está elegida la placa NodeMCU 1.0	Desconecta el HC-05 (al menos `TXD`) y revisa *Herramientas* → *Placa*
Aparecen caracteres extraños en el Monitor Serie al encender	Es el mensaje de arranque del ESP8266, enviado a otra velocidad	Es normal: ignóralo. Si todo lo que ves es ilegible, ajusta el Monitor Serie a 9600 baudios
Un motor o sensor no responde al cargar el código	Cable en el pin equivocado, o alimentación desconectada	Vuelve al Capítulo 4 y revisa esa etapa específica antes de tocar el código
El robot avanza cuando debería retroceder	Motor cableado con polaridad invertida	Intercambia los dos cables de ese motor, no cambies el código
Un sensor dice `LINEA` sobre el blanco y `blanco` sobre la cinta	El módulo entrega el nivel contrario al que supone el programa	Cambia `NIVEL_LINEA` de `LOW` a `HIGH` (o al revés)
Las lecturas de los sensores cambian con la luz del lugar	La luz solar o de lámparas cálidas contiene infrarrojo	Calibra de nuevo en el lugar de uso y evita el sol directo sobre la pista

Tabla B.4. Comunicación (Capítulo 6).

Síntoma	Causa probable	Solución
El celular no encuentra el HC-05	Módulo sin alimentación, o fuera de rango	Verifica el LED del HC-05 (parpadeo rápido = listo para emparejar)
La app no logra conectar al módulo	Emparejamiento Bluetooth del celular no completado antes de abrir la app	Empareja primero desde los ajustes del celular, con el código `1234` o `0000`
La app no se instala o no encuentra el Bluetooth	Celular iPhone, o apps de origen desconocido bloqueadas	Usa un celular Android y permite instalar el `.apk`
La app dice «Bluetooth no conectado»	Se tocó un botón antes de conectar	Toca «Conectar BT» y elige el HC-05
La app conecta, pero el robot no reacciona	Líneas de datos sin cruzar (`TX` con `TXD`), o velocidad del módulo distinta de 9600	Conecta `TXD`→`RX` y `RXD`→`TX`. Si cambiaste la velocidad del HC-05 con comandos AT, usa esa misma en `Serial.begin()`
El control manual no mueve el robot	El mensaje no empieza con `M:` o no lleva dos números separados por una coma	Revisa en Bloques el texto que se envía, contra la Tabla 6.2, y en el programa la función `procesarDatos()`
El robot sigue moviéndose al soltar el control	La app no manda `M:0,0` cuando no tocas nada	Revisa que el temporizador (`Clock`) esté habilitado en la pantalla «Manual» y que mande valores cero al soltar

Tabla B.5. Seguidor de línea y PID (Capítulo 7).

Síntoma	Causa probable	Solución
«Enviar» no pone en marcha el seguidor	Está cargado el programa del Capítulo 6, que no incluye el seguidor	Sube `CodigoDefinitivo.ino` (Código 7.1)
El robot gira en el lugar o se aleja de la línea	Los niveles `LOW`/`HIGH` de los sensores están invertidos, sus cables de señal están intercambiados, o un motor está cableado al revés	Repite la calibración del Capítulo 5 y la prueba de movimientos, e intercambia los cables que corresponda
Se sale en las curvas	`KP` muy bajo, velocidad muy alta o curva muy cerrada	Sube `KP` o baja `VELOCIDAD_BASE`; abre la curva
Zigzaguea en las rectas	`KP` muy alto	Baja `KP`; prueba un `KD` pequeño
Rinde distinto cada día	Pilas más o menos cargadas, o distinta luz en la pista	Prueba con pilas nuevas y calibra los sensores en el lugar

C Archivos descargables

El código definitivo del seguidor de línea, la app de control, el modelo 3D del chasis del autor y este mismo libro en PDF están disponibles para descarga libre en el repositorio:

`https://github.com/EduardoFuentesUA/IntroRobotica`

Los de esta guía están en la carpeta `guia-1-introduccion-a-la-robotica/`. Para bajar todo de una vez, usa el botón **Code** y luego **Download ZIP** en la página del repositorio; no necesitas una cuenta de GitHub.

Tabla C.1. Archivos de esta guía en el repositorio.

Carpeta o archivo	Contenido
`Introduccion-a-la-Robotica.pdf`	Este libro
`codigo/CodigoDefinitivo/`	Código definitivo del seguidor de línea, con control PID y modo manual (Capítulo 7)
`app/Guia_Robotica.aia`	Proyecto de la app, para abrir y editar en MIT App Inventor (Capítulo 6)
`app/Guia_Robotica.apk`	App lista para instalar en un celular Android (Capítulo 6)
`chasis/chasis.stl`	Placa del chasis del autor (Capítulos 2 y 3)
`chasis/guardabarros.stl`	Guardabarros del chasis del autor
`chasis/union-motor-1.stl`	Pieza de unión de los motores al chasis
`chasis/union-motor-2.stl`	Segunda pieza de unión de los motores

NOTA

El programa está dentro de una carpeta con su mismo nombre, porque el Arduino IDE así lo exige. Abre el archivo `.ino` desde esa carpeta, sin moverlo ni renombrarlo, y súbelo con la placa **NodeMCU 1.0 (ESP-12E Module)** elegida (Capítulo 5). Los demás programas del libro (Capítulos 5 y 6) no se descargan: están completos en sus capítulos, para que los escribas tú mismo. Si modificas la app en App Inventor, exporta de nuevo tanto el `.aia` (para seguir editando) como el `.apk` (para instalar).

ATENCIÓN

La placa del chasis del autor mide 250×130 mm y no cabe en las camas de 220×220 mm de muchas impresoras. Antes de laminarla, comprueba en Cura que quede dentro del volumen de impresión. Si no cabe, úsala como referencia y diseña tu propia placa, como propone el Capítulo 2.

Bibliografía

[1] Arduino. Arduino Documentation. https://docs.arduino.cc/. Consultado el 2 de octubre de 2026.

[2] Arduino. SoftwareSerial Library Reference. https://docs.arduino.cc/learn/built-in-libraries/software-serial/. Consultado el 2 de octubre de 2026.

[3] Karl J. Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2 edition, 2021.

[4] Guangzhou HC Information Technology Co., Ltd. HC Serial Bluetooth Products: User Instructional Manual. https://components101.com/sites/default/files/component_datasheet/HC06%20Bluetooth%20Module%20Datasheet.pdf. Manual del fabricante, copia alojada por Components101. Consultado el 2 de octubre de 2026.

[5] MIT. MIT App Inventor Support. https://appinventor.mit.edu/explore/ai2/support/.

[6] Onshape. Onshape Help. https://cad.onshape.com/help/. Consultado el 2 de octubre de 2026.

[7] STMicroelectronics. L298 Dual Full-Bridge Driver. https://www.st.com/resource/en/datasheet/l298.pdf. Hoja de datos del fabricante.

[8] UltiMaker. UltiMaker Cura. https://ultimaker.com/software/ultimaker-cura/. Consultado el 2 de octubre de 2026.

[9] Vishay Semiconductors. TCRT5000, TCRT5000L: Reflective Optical Sensor with Transistor Output. https://www.vishay.com/docs/83760/tcrt5000.pdf. Hoja de datos del fabricante.